Tunisian Republic Ministry of Higher Education And Scientific Research

University of Sfax

The National School of Electronics and Telecommunications of Sfax

Engineer in :
Engineering of Electronic Communication Systems

Option : Connected Systems

End of Studies Project

Order N° : GEC-SC-3-2025-026

OIP -FR-01-00
05/05/2025

Graduation Report

Presented at

The National School of Electronics and Telecommunications of Sfax

In order to obtain the

Engineering Diploma in:
Engineering of Electronic Communication Systems

Option :
Connected Systems

By

Ezzeddine Rayene

Web Application to visualize Network Topology

Presented on 14/ 06/ 2025, before the review panel

Mr.	**Hajji Soufiane**	President
Mr.	**Neji Mohamed**	Examiner
Mr.	**Samaoui Souhail**	Academic Supervisor
Mrs.	**Ayadi Raouia**	Academic Supervisor
Mrs.	**Moalla Rawia**	Industrial Supervisor

Dedication

First and foremost, as I complete this internship project, I would like to thank God for blessing me with health and well-being.

To my parents, Mohamed Ali Ezzeddine and Sourour Hadj Sassi

No words can truly express the depth of love and respect I feel for them. I thank them from the bottom of my heart for the education, well-being, and unconditional support both moral and material that they have always provided. Their sacrifices and prayers have been the foundation of my journey. I dedicate this work to them with the hope that I make them proud. May God protect them and grant them health and happiness

To my sister, Zeineb Ezzeddine

I wish you a life full of happiness and success, and that you become a great engineer. Thank you for your constant support and encouragement.

To all my friends

In memory of our friendship and the wonderful moments we have shared together.

Rayene Ezzeddine

Acknowledgments

An internship is not merely an additional step in a student's academic journey; it is a framework for new experiences gained daily, guided by those who support and inspire us.

With heartfelt and sincere gratitude, I would like to thank all those who, in their own way, helped me successfully complete this internship.

First and foremost, I would like to express my deepest respect and appreciation to **Mrs. Rawia Moalla** for her patient guidance and high-quality supervision. I am truly grateful for her trust and valuable support throughout this experience.

I would also like to thank the entire team at KPIT. Each member has been supportive and always willing to assist me in completing my tasks.

Furthermore, I would like to express my sincere gratitude to my academic supervisors, **Mrs. Raouia Ayadi and Mr. Souhail Smaoui**, for their unwavering support and guidance during the internship period. I also extend my thanks to all the professors at the National School of Electronics and Telecommunications of Sfax (ENETCOM), especially in the Electronics Department, for their dedication and commitment to student development.

Finally, I would like to extend my heartfelt thanks to the jury members, **Mr. Mohamed Neji and Mr. Soufiane Hajji** , for their time and willingness to evaluate my work as part of this jury.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AUTOSAR Automotive Open System Architecture

ARXML AUTOSAR XML

ADAS Advanced Driver Assistance Systems

AI Artificial Intelligence

BSW Basic Software

CAN Controller Area Network

CAN-FD Flexible Data-Rate

DB Data Base

DRF Django REST Framework

ECU Electronic Control Units

HiL Hardware In the Loop

HTTP Hypertext Transfer Protocol

IP Internet Protocol

JSON JavaScript Object Notation

LIN Local Interconnected Network

LLaMA Large Language Model Meta AI

LLM Large Language Model

LoRA Low Rank Adaptation

MOST Media Oriented Systems Transport

MBD Model Based Design

MUI Material User Interface

OTA over-the-air

ORM Object Relational Mapper

PDU Protocol Data Unit

RTE Runtime Environment

STLC Software Testing Lifecycle

SOME/IP Scalableservice-Oriented Middleware over IP

SQL Structured Query Language

XML Extensible Markup Language

GENERAL INTRODUCTION

As modern vehicles become increasingly software-driven, the complexity of their underlying systems has grown significantly. Automotive software now manages everything from safety-critical functions to infotainment features, requiring seamless coordination between numerous electronic control units (ECUs).

Automotive Open System Architecture (AUTOSAR) is one of the most widely adopted frameworks in automotive software development. It provides a standardized approach to software design and integration. However, the ARXML files used within AUTOSAR are often large and highly complex, containing vast amounts of configuration data that can be difficult to interpret and navigate ,particularly for testing and validation teams.

To support this challenge, our project focuses on facilitating the work of testers by providing a web-based visualization tool that makes AUTOSAR system structures and networking more accessible and understandable.We began our work from a pre-converted database (DB file) in which the relevant software elements were already structured and available. This allowed us to concentrate on processing and presenting the data in an intuitive, user-friendly way.

The application provides a visual representation of communication between essential automotive- software components, including ECUs, services, methods, events, and signals. Its core purpose is to help testers better understand the architecture of in-vehicle communication network such as Controller Area Network (CAN) and Ethernet. By offering a clear, interactive overview of how these components are interconnected, the tool enhances visibility and makes it easier to trace communication paths. Additionally, it features a chatbot that allows testers to query the uploaded file and receive relevant answers, helping to establish a strong foundation for creating accurate and effective test cases This report will be structured as follows:

Chapter 1 will provide a comprehensive project overview, clearly defining its core objectives and strategic rationale.

Chapter 2 will present a preliminary study establishing the technical foundation and automation framework required for development.

Chapter 3 will transition into system design, featuring detailed UML diagrams that will guide the architectural implementation.

Finally, Chapter 4 will focus on practical implementation, examining the application architecture, user interface, system integration processes, and providing an in-depth analysis of the AI model deployment.

Chapter

1 General Context of the project

Contents

1.1 INTRODUCTION

This chapter provides an overview of the project's background and its organizational context. It begins with a presentation of the hosting organization, outlining its core values and key activities to establish the relevance of the project within its operational framework.

It then examines the context in which the project was initiated, including a review of the current validation process, identification of limitations, and the rationale behind the proposed solution. The chapter concludes with a brief description of the methodological approach, highlighting the use of Agile practices,specifically the SCRUM framework,to guide the project's execution.

1.2 Presentation of the hosting organization

The project, titled "Developing a Web Application to Visualize Ethernet Topology", is part of the final internship for obtaining a National Engineering Degree in Electronics and Communication Engineering, with a specialization in Connected Systems, from the National School of Electronics and Telecommunications of Sfax. This project was conducted over a four-and-a-half-month period at KPIT Technologies.

1.2.1 Presentation of KPIT

KPIT Technologies is a global technology company based in Pune, India, with a strong presence in several countries, including Germany, the USA, Japan, and more [1]. Established to drive innovation in mobility, KPIT has grown into a leading provider of software integration, embedded systems, and engineering services tailored to the automotive and mobility sectors. With over 10,000 professionals worldwide, KPIT is known for its deep domain expertise, particularly in electric and autonomous vehicles.

Figure 1.1: KPIT's logo [1]

1.2.2 KPIT's Values

KPIT's corporate philosophy is built on foundational pillars that shape its industry partnerships and technological leadership and :

- **Engineering Excellence:**Committed to innovation, KPIT develops cutting-edge software solutions for future mobility, with particular focus on sustainability, safety, and software -driven development.
- **Comprehensive Service Portfolio:** KPIT offers end-to-end services, including software development, integration, validation, and maintenance for embedded systems, particularly in the automotive domain.
- **Strategic Global Alliances:** KPIT maintains close collaborations with major OEMs and Tier 1 suppliers, providing expertise in domains such as Autonomous Driving, Electric Powertrain, Connected Vehicles, and Vehicle Diagnostics.

1.2.3 KPIT Architecture

As presented in Figure 1.2, KPIT follows a hierarchical organizational structure designed to ensure clear roles, efficient decision-making, and streamlined operations. This framework enables effective collaboration across teams while aligning with strategic goals.

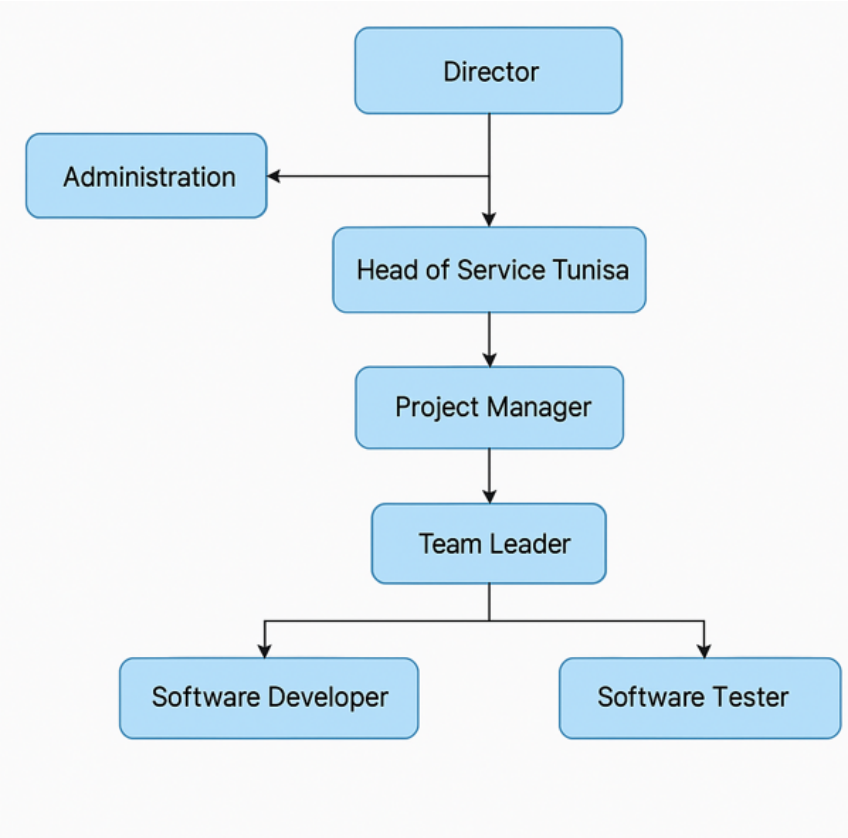

Figure 1.2: KPIT Architecture

1.2.4 KPIT's Core Activities

As a technology leader in automotive solutions, KPIT drives innovation through these key operational domains:

- **Embedded Software Development:** KPIT develops software for ECUs, focusing on AUTOSAR, adaptive platforms, and real-time systems for various vehicle domains.
- **Model-Based Design and Validation:** The company leverages model-based development (MBD) for software design and performs rigorous validation using hardware-in-the-loop (HiL) and software-in-the-loop (SiL) environments.
- **Electrification and Autonomous Systems:** KPIT actively contributes to the development of technologies for electric vehicles (EVs) and autonomous driving systems, supporting innovation in Advanced Driver Assistance Systems (ADAS) and battery management.

- **Diagnostics and Connectivity Solutions:** KPIT provides robust diagnostics platforms and connected solutions, enabling real-time vehicle data monitoring and over the air (OTA) updates.
- **Collaboration with OEMs and Tier 1 suppliers:** The company partners with global automotive giants such as BMW, Daimler, Ford, and Continental to deliver scalable, reliable solutions tailored to industry needs.

1.3 Project presentation

In this section, we will present the existing validation process, critique the current solution, and conclude with the proposed solution.

1.3.1 Study of the existing Validation Process

As part of the analysis of the existing validation process, it is essential to understand the methodological foundations on which any testing approach in the automotive industry is based. One of the most widely adopted and rigorous frameworks for quality assurance is the Software Testing Lifecycle (STLC). This cycle provides a structured approach designed to ensure software quality through a series of well-defined phases, ranging from planning to test closure. Each phase plays a critical role in identifying and correcting defects before software deployment. The STLC thus represents the backbone of software validation in demanding environments such as the automotive sector as shown in Figure 1.3. At KPIT, this methodology is built upon core principles: complete traceability of requirements, ensuring that each functional need is validated, optimized test coverage using statistical tools, ensuring efficiency without redundancy, and automated documentation of results, essential for regulatory compliance and audit readiness. The key phases of this cycle are presented in Table 1.1:

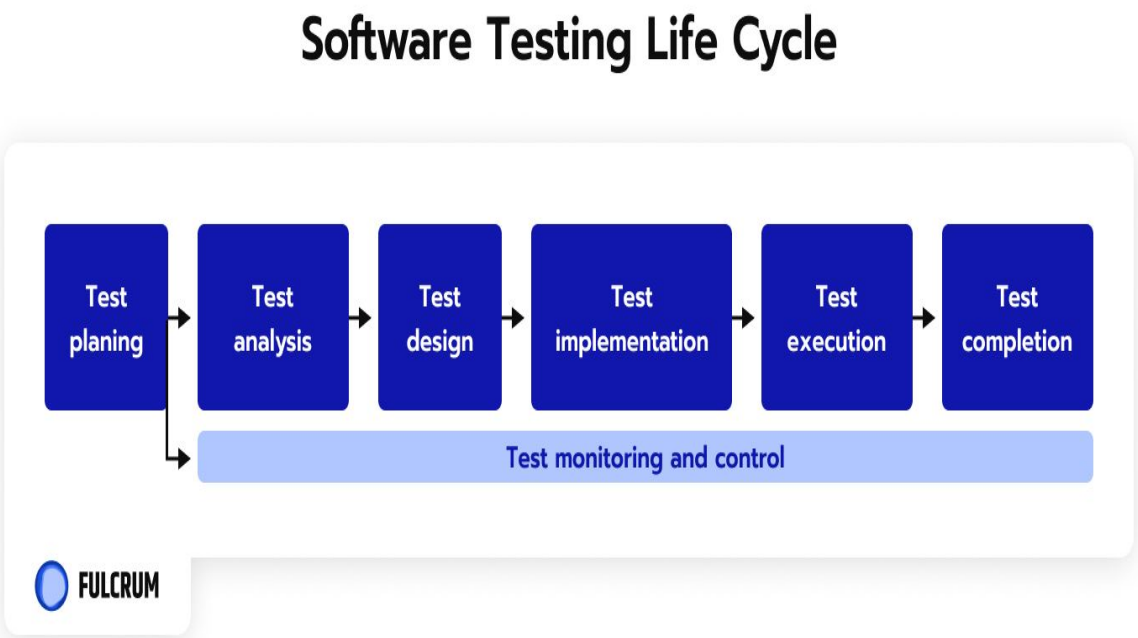

Figure 1.3: Software Testing Life Cycle[2].

Table 1.1: Software Testing Lifecycle Phases

Phase	Main Focus	Key Activities
Analysis	What to test	Review requirements, identify test conditions, clarify ambiguities
Design	How to test	Create test cases, define test data, and plan expected results
Implementation	Prepare for testing	Develop test scripts, set up test environment and tools
Execution	Perform the tests	Run test cases, log defects, and perform re-testing after fixes

The project described in this report fits directly within the design phase of the STLC. This phase is crucial, as it defines the strategies, tools, and artifacts needed to generate test cases and achieve the expected functional coverage. It lays the technical groundwork for all downstream validation activities. The study and automation of ARXML file analysis, central objects in AUTOSAR-based architectures, serve as a foundational input to this design phase, delivering structured, reliable, and traceable data essential for test design and planning.

In this context, KPIT Technologies integrates a specialized toolchain into its validation workflow, with ANDI being a key proprietary solution developed for the analysis of ARXML files. These files, generated in compliance with the AUTOSAR standard, are critical components in the configuration of automotive embedded systems. They encapsulate complex and highly structured data, including ECU specifications, signal definitions, network topologies, and communication parameters. Ensuring the integrity and consistency of these files is vital to the reliability of the generated test cases in later stages of the STLC.

To meet this requirement, ANDI offers advanced syntactic and semantic analysis capabilities. It enables automatic resolution of cross-references between AUTOSAR objects, and the generation- of technical metadata that can be directly used in test case generation or optimization. A typical ARXML file may contain more than 200 types of business objects, organized into complex hierarchical and cross-linked structures. In this setting, a robust tool like ANDI is essential for ensuring reliable exploitation of this data during test design.

Moreover, due to their significant size and nested complexity, analyzing ARXML files manually or in raw format is extremely resource- and time-intensive. To address this challenge, KPIT has developed innovative internal solutions that convert ARXML content into more accessible- formats, particularly relational databases. This transformation allows for targeted data access, faster processing, and seamless integration with test automation and analytics.

This strategy not only improves the operational efficiency of the validation process but also reinforces the structuring of input data at the early design stage. Automating the analysis of ARXML files serves as a key enabler for making informed design decisions and achieving comprehensive test coverage. A typical example of an ANDI script output is shown in Figure1.4, illustrating how AUTOSAR objects and extracted metadata are leveraged to support automated validation .

To overcome these challenges, KPIT has developed internal tools to transform ARXML files into more accessible formats, such as relational databases, to enhance processing efficiency. This transformation improves the validation workflow by allowing engineers to query, visualize,

Figure 1.4: Script Output Andi.

and verify communication topologies and configurations more effectively without the overhead of parsing XML during runtime.

1.3.2 Critique of the Existing Approach

Despite the notable improvements brought by the development of DataBex, the overall validation workflow still encounters significant limitations that hinder its full efficiency. While the ARXML -to-database conversion undeniably accelerates data access by eliminating the need for direct parsing of large XML files, it only partially addresses the broader challenges associated with data exploration, usability, and user interaction.

One of the most persistent bottlenecks lies in the visualization phase. Currently, testers must rely on DB Browser to explore the content of the generated .db files. However, this tool, originally designed for generic database inspection, is not well suited to the specific demands of automotive data validation. Its interface is technical, minimalistic, and non-intuitive, particularly for users without strong backgrounds in database querying. As a result, testers often face difficulties locating relevant information, tracing signal paths across multiple tables, or identifying- relationships between communication objects.

These challenges are exacerbated by the lack of integrated features such as advanced filtering, keyword-based search, hierarchical navigation, or domain-specific visual cues. Moreover, DB Browser does not provide context-aware metadata, guided workflows, or automotive protocol-specific representations, which are essential for efficiently interpreting AUTOSAR data structures. This absence of specialized support forces testers to spend considerable time

manually navigating, cross-checking, and interpreting the raw content of the database—often leading to confusion, delays, and increased risk of human error.

Consequently, although DataBex contributes to a noticeable reduction in raw processing time, the overall user experience and productivity remain suboptimal. The validation process continues to feel slow, fragmented, and error-prone, as testers are burdened with low-level data manipulation tasks that could otherwise be streamlined through intelligent visualization

1.3.3 Proposed Solution

To streamline the work of testers, particularly during the analysis, design, and implementation phases of the STLC, we propose the development of a dedicated web application for the automa-ted- extraction and visualization of data derived from ARXML files converted into relational database format. This solution is intended to overcome the limitations of existing tools, such as their lack of user-friendliness, absence of graphical representation, and the difficulty in exploring complex relationships between components in automotive embedded systems.

The proposed application will begin by extracting the needed data from the DB file to automatically identify and structure key relationships between core system elements, including ECUs, provided or consumed services over the Ethernet protocol, and transmitted or received communication frames. This information will be presented through a first interactive diagram, designed to support intuitive and visual exploration of th dependencies and system's internal logic.

A second diagram will illustrate interactions between ECUs, coupling ports, and Virtual Automotive Local Networks (VALNs), offering a clear overview of the system's network topolo -gy-. This dual visualization, focused on both functional and communication layers.

Through a user-friendly web interface, users will be able to upload database files, apply dynamic filters to isolate specific components or data flows, and navigate interactively through the rendered diagrams. This approach significantly enhances the accessibility, clarity, and accuracy of analyzing complex system architectures, whether based on CAN or Ethernet protocols.

To further support the validation process, the application will feature a test code generation module that automatically produces test scripts from the extracted diagrams. These scripts will be used to verify that each ECU transmits and receives the correct messages in alignment with its defined configuration. This automated generation reduces manual test writing effort, ensures consistency with the architecture model, and strengthens traceability throughout the STLC.

In addition, the platform will integrate an intelligent chatbot assistant, designed to facilitate real-time data interaction and support. This AI-powered chatbot will allow users to ask natural language questions about the uploaded database. For example, a user might ask, “Which ECUs are part of VALN-3?” or “What frames are consumed by ECU-B over Ethernet?” The chatbot will interpret these queries, convert them into SQL commands, execute them, and return the results in a structured and readable format, often linked directly to relevant diagram elements.

By combining automated visualization, test generation, and AI-assisted querying, this web application aims to significantly improve tester productivity, reduce errors, and simplify the analysis of large-scale automotive systems. Ultimately, it provides a comprehensive, modernized framework for managing the increasing complexity of validation in AUTOSAR.

1.4 Working Methodology

In order to deliver a high-quality solution, it was essential to operate with maximum efficiency. To achieve this, we adopted an Agile methodology for the execution of the project, enabling optimal management and continuous improvement throughout its development.

1.4.1 Agile methodology

To ensure quality, flexibility, and efficiency in the development of the proposed web application, we adopted an Agile approach, ideally suited for a dynamic and user-centered project. This approach allowed us to respond quickly to changes, integrate complex features such as interactive visualizations and the chatbot assistant, and ensure continuous improvement of the product at each iteration.

Agile methodologies are project management strategies that rely on iterative and incremental cycles [3], allowing the development process to respond effectively to user feedback, evolving requirements, and technical limitations.

1.4.2 Agile Method: SCRUM

Our team follows the Agile methodology known as SCRUM, which structures the project into time-boxed iterations called Sprints, typically lasting one to four weeks. Each Sprint starts with an estimation phase and an operational planning session, and concludes with a demonstration of the completed work. Prior to initiating the next Sprint, the team conducts a retrospective to evaluate the previous cycle and identify opportunities for improvement. This process is illustrated in Figure 1.5 [4]. The development team operates through self-organization The choice of this methodology was based on several key advantages:

- It enables better communication, as users can continuously clarify and prioritize their requirements.
- The client has greater visibility into the progress of the project.
- Quality is well controlled through continuous testing
- Risks can be identified and addressed at an early stage.
- The team remains motivated, aiming to build trust and meet the defined objectives.
- Cost control is improved.

-SCRUM Roles and Responsibilities

In a SCRUM framework, each role has distinct responsibilities that contribute to the success of the project. The table 1.2 outlines the main roles in SCRUM and their respective duties.

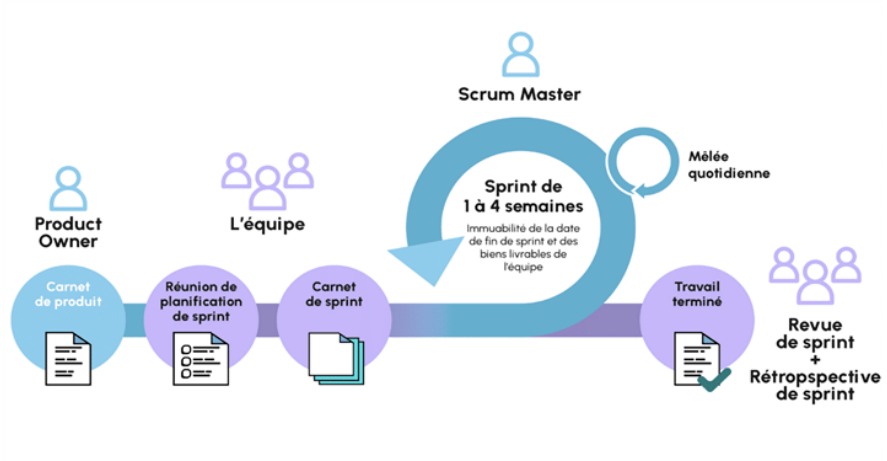

Figure 1.5: SCRUM agile methodology [4]

Table 1.2: SCRUM Roles and Responsibilities

Roles	**Responsibilities**
Product Owner	The official representative of the client within a SCRUM project. They define and prioritize the product requirements.
SCRUM Master	This person ensures that the SCRUM methodology is properly implemented and that its objectives are met. They are responsible for facilitating communication. Although not a project manager, the SCRUM Master removes any obstacles that may hinder the team's progress during the Sprints. They play a key role in ensuring smooth execution and resolving impediments.
SCRUM Team	Responsible for executing the Sprints and delivering the outputs. The team must be autonomous and capable of meeting client requirements and honoring its commitments.

-SCRUM Events

Several key meetings are implemented to ensure effective communication among SCRUM stakeholders:

- **Sprint Planning Meeting:**

  This is a crucial meeting held at the beginning of each Sprint. The development team and the Product Owner collaborate to define the goals for the upcoming Sprint, based on the prioritized *Product Backlog*. They also estimate the production capacity required to meet these objectives.

- **Daily SCRUM (Stand-Up Meeting):**

  Every day, the project team gathers for a short meeting (around 15 minutes). Each member shares what they worked on the previous day, any obstacles encountered, and what they plan to accomplish on the current day. This promotes alignment and transparency within the team.

- **Sprint Review Meeting:**

  At the end of the Sprint, the team presents the completed functionalities to the Product Owner and possibly end users. Feedback is collected, and the scope of future Sprints is discussed. Adjustments may be made to the overall release planning based on progress and priorities.

- **Sprint Retrospective:**

  This meeting also takes place at the end of each Sprint and is facilitated by the SCRUM Master. The team reflects on their collaboration, performance, and challenges encountered during the Sprint. The goal is to identify opportunities for improvement and enhance future Sprints.

-SCRUM Artifacts

SCRUM also relies on specific communication artifacts that guide the development process:

- **Product Backlog:**

  This is a dynamic list of all the features and functionalities desired by the client, typically written as *User Stories*. It is visible to all stakeholders, who may suggest new items. The Product Owner continuously updates and prioritizes the backlog items.

- **Sprint Backlog:**

  This contains the set of items selected for the current Sprint. These are defined Collectively by the Product Owner and the development team during the Sprint Planning meeting. The Sprint Backlog is visible to the entire team and serves as a reference during Daily SCRUMs.

1.4.3 Project planning

Our project was carried out progressively, following a strict chronological order. The figure 1.6 presents the planning of the tasks to be completed throughout the project. This schedule allows us to assess our progress at each stage in relation to the timeframes defined in the calendar. This is one of the key advantages of the Agile methodology, as it ensures that the final product will be complete, deliverable, and of acceptable quality.

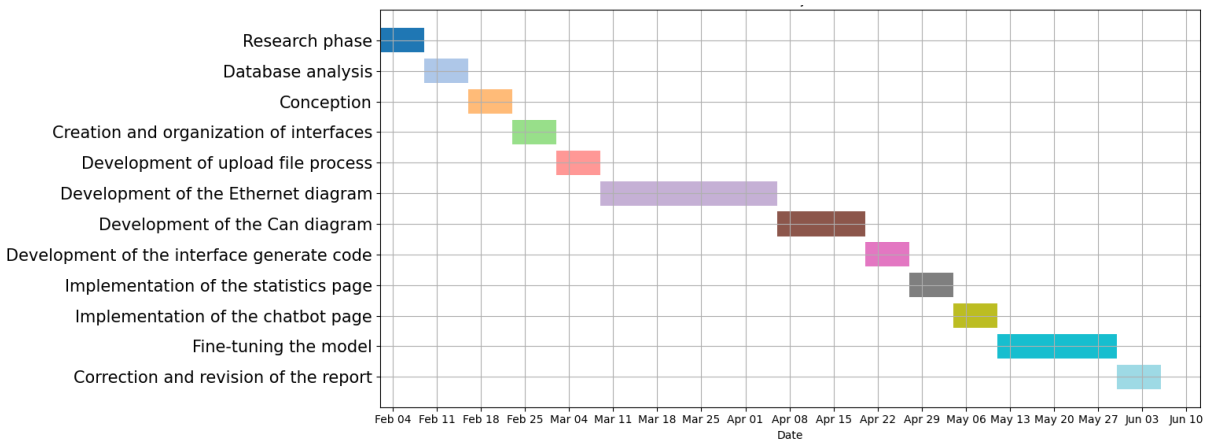

Figure 1.6: Diagram of Gantt.

1.5 CONCLUSION

This chapter provided a comprehensive overview of the project's foundational elements. It began with an introduction to the hosting organization, highlighting its core values, mission, and operational scope. Following this, the chapter delved into the context of the project, analyzing the current validation process and identifying key limitations in the existing approach. A proposed solution was outlined to address these challenges effectively.

Lastly, the chapter introduced the working methodology, with a focus on Agile practices and the SCRUM framework. This methodological choice was justified by its adaptability, iterative nature, and emphasis on collaboration. These insights establish the framework for the following chapters, which will delve deeper into the implementation and results of the proposed solution.

Chapter

2 Automotive Network and SQL Coder

Contents

2.1 INTRODUCTION

This chapter delves into the foundational aspects of vehicle network architecture, establishing the framework for a comprehensive understanding of modern automotive systems. We begin with a preliminary study, providing an overview of the automotive industry and highlighting the crucial role of the ECU. Furthermore, we introduce AUTOSAR, a standardized software architecture that plays a significant role in automotive development.

Following this introduction, we explore essential technical concepts, analyzing various types of automotive network architectures, including the well-established CAN bus system and the increasingly prevalent Ethernet SOME/IP. In addition, this chapter discusses the concept of DB files within vehicle network architecture and their importance in automotive testing and validation, particularly in generating test cases for ECU testing.

Finally, we explain the model we use and present the rationale for choosing Defog SQL Coder to be integrated as a chatbot within the system.

2.2 Preliminary Study

This section offers a foundational understanding essential to the project by exploring key aspects of the automotive industry. It covers the industry's current landscape, the role of ECUs, and introduces the AUTOSAR standard as a framework for automotive software architecture .

2.2.1 Overview of the Automotive Industry

The automotive industry comprises a diverse array of businesses engaged in the design, development-, manufacturing, marketing, sales, repair, and modification of motor vehicles. As a major global sector, it is recognized for its substantial revenue generation and heavy investment in research and development. The term automobile, denoting self-powered vehicles, first appeared- in the New York Times on January 3, 1899 [5]. Automotive production plays a crucial role in transportation and makes a significant contribution to economic growth.

Recent developments in autonomous driving technology have significantly improved transportation- efficiency, convenience, and safety. As modern vehicles integrate a growing number of advanced features, software has become increasingly central to their functionality, marking a shift in the industry's emphasis from mechanical systems to electronics and software. This evolution is reflected in the widespread adoption of graphic interfaces and integrated displays in contemporary vehicles [6].

Automation systems,particularly fully autonomous vehicles,are becoming increasingly popular- due to their safety advantages. Advances in machine learning and artificial intelligence have made these technologies vital across multiple sectors, often replacing human involvement in key functions. As global demand for safe, efficient, and sustainable transportation grows, the automotive industry plays a pivotal role in addressing these needs and is actively investing in innovations to support long-term sustainability goals [7].

2.2.2 Electronic Control Unit

ECUs are advanced embedded systems designed to handle various input and output functions as shown in figure 2.1, ensuring the efficient operation of modern vehicles. These units process data from multiple ,such as temperature, pressure, and speed sensors,while also interpreting user commands. By analyzing this information, ECUs make real-time decisions to regulate actuators, including electric motors, solenoid valves and lighting systems.

In addition to their primary function in vehicle control, ECUs play a vital role in enhancing safety and enabling diagnostics. They monitor potential faults in sensors and actuators, recording errors to facilitate troubleshooting and maintenance. With the growing technological complexity of modern vehicles, the number of ECUs per vehicle has risen significantly. These units communicate via various automotive networks, including CAN, Local Interconnect Network (LIN), FlexRay, Media Oriented Systems Transport (MOST), and Ethernet [8]

Given the increasing complexity of vehicle electronics, industry leaders and standardization bodies are working towards unified protocols to improve interoperability and efficiency.This is especially evident in initiatives such as AUTOSAR, which seeks to standardize ECU software

architectures across different automotive manufacturers [9]. By optimizing communication between ECUs, these advancements contribute to enhanced vehicle performance, reliability and safety.

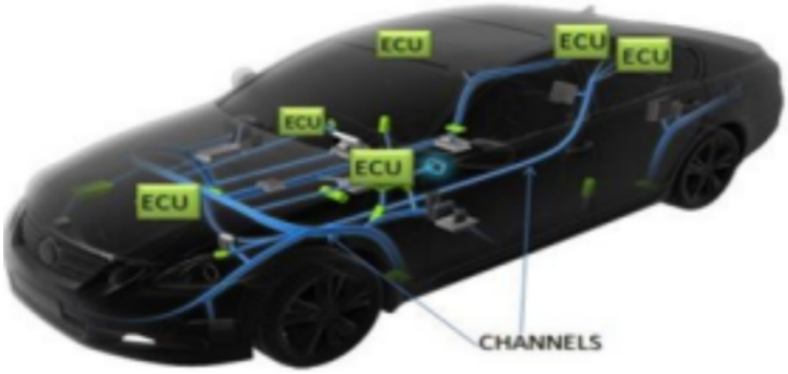

Figure 2.1: Example of ECU composition in a vehicle [10] .

2.2.3 AUTOSAR

AUTOSAR is a standardized framework in the automotive industry designed to address the complexity and interconnectivity of modern vehicle electronic systems. It defines a three-layered architecture, which consists of the following:

- **Application Layer:** This top layer comprises vehicle-specific applications, such as power-train- management ,body electronics, and infotainment systems [12].
  It ensures interoperability among different ECUs through standardized interfaces and communication protocols, as defined by standards like AUTOSAR
- **The Runtime Environment (RTE)** functions as a bridge between the application layer and the Basic Software (BSW) layer, enabling standardized communication, seamless data exchange, inter-ECU interactions, event handling, and efficient real-time task scheduling.
- **Hardware Layer** Serving as the backbone of the architecture, this layer comprises physical ECUs and microcontrollers responsible for executing the software. AUTOSAR accommodates diverse hardware platforms, ensuring seamless implementation of standardized- software across various hardware configurations.

The AUTOSAR standardized framework fosters collaboration and interoperability among stakeholders in the automotive industry, enabling the seamless integration of software components- from various suppliers. By providing a common development language and structure, AUTOSAR streamlines software updates, lowers development costs, and accelerates time-to-market for new features. This standardization enhances the scalability, flexibility, and reliability of automotive systems, supporting the creation of advanced vehicle functionalities and delivering a smooth user experience. Figure 2.2 illustrates the AUTOSAR architecture.

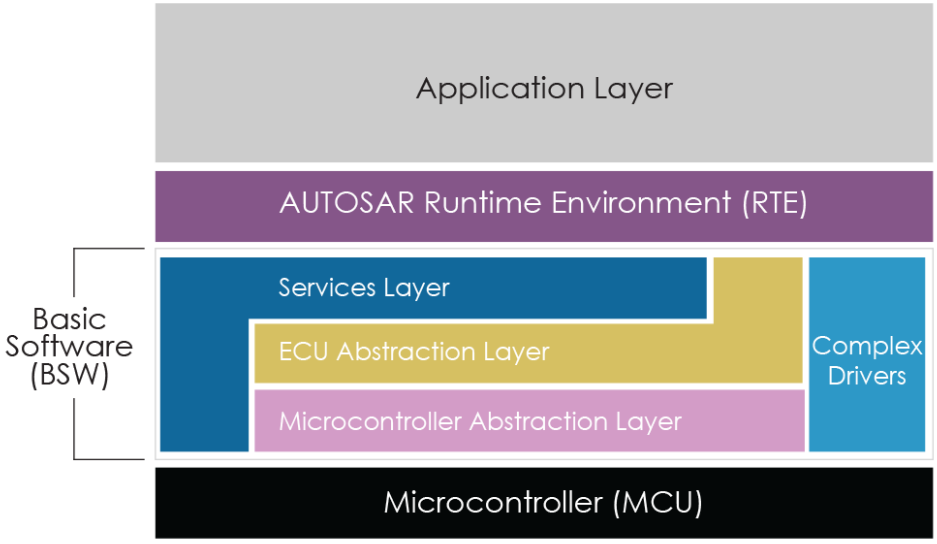

Figure 2.2: Autosar architecture [11] .

2.3 Basic Technical Concepts

This section outlines key automotive network architectures, including Ethernet SOME/IP and CAN, which enable in-vehicle communication and are central to our project.

2.3.1 Types of Automotive Network Architectures

Communication networks within vehicles are crucial for the efficient and safe operation of all electronic systems. Modern cars integrate several types of in-vehicle networks, each serving specific functions

- **Ethernet**: Used for high-speed communication, including video, voice, internet data, OTA updates, and diagnostics.
- **CAN** : is a serial network information technology that facilitates the passing of information between Electronic Control Units Enables real-time communication between various sub-systems- such as the engine, transmission, and braking systems.
- **CAN-FD (Flexible Data-Rate CAN)**: An enhanced version of CAN that supports higher bandwidth and larger data payloads.
- **FlexRay**: Designed for safety-critical applications that require high-speed and deterministic- communication, such as steering and braking systems.
- **Local Interconnect Network (LIN)**: A cost-effective, low-speed network used for communication- between simple electronic components like sensors and actuators.

2.3.2 Ethernet SOME/IP

Scalable service-Oriented Middleware over Internet Protocol (SOME/IP) is a communication protocol designed to facilitate service-oriented communication within automotive Ethernet networks- [13]. It enables the transmission of complex data types using mechanisms such as remote procedure calls (RPC), notification events, and fields. Communication and related information are provided through services and their respective service interfaces [13].

a) Architecture and Communication Mechanism

SOME/IP operates over IP networks, specifically leveraging Ethernet networks in the automotive domain. The architecture of SOME/IP consists of several key components:

- **Service Consumer:** This is the entity that requests a service.
- **Service Provider:** This entity provides the requested service.

- **Service Discovery:** This component enables the identification of available services and their communication endpoints within the network.

The communication mechanism follows a client-server model. A client (service consumer) sends a request to a server (service provider) via the SOME/IP protocol. The communication typically involves a request-response pattern, where the client waits for a response from the server. However, SOME/IP also supports asynchronous and event-based messaging for more complex interactions.

b) Service Components

The SOME/IP architecture defines several key components that facilitate service-oriented communication-. These components are essential for enabling efficient and reliable communication between different entities within a distributed system.

-Events: are notifications transmitted by the Service Provider to the Subscriber, designed to convey changes or updates. These notifications can be categorized as:

- *Cyclic Events*: Sent at predefined, fixed intervals.
- *Triggered on Change Events*: Sent when a specific condition is met or a state transition occurs.

-Eventgroups: are logical aggregations of multiple events, allowing Subscribers to manage subscriptions to related events as a single unit, thus improving efficiency.

-Methods: represent operations or Remote Procedure Calls (RPCs) executed by the Service Provider. A Subscriber can invoke a method to request a specific operation or retrieve data, method invocation adheres to a request-response model in synchronous communication. The Subscriber sends a request, and the Provider executes the operation, returning the result.

-Fields: represent attributes or data points of a service, which can be read, written, or monitored by the Subscriber. Fields may reflect various system states, such as sensor readings
or configuration parameters.Fields serve multiple roles:

- *Notifier*: The Service Provider sends updates when the field's value changes, reflecting dynamic system states.
- *Getter*: A Subscriber can query the Service Provider for the current field value.
- *Setter*: A Subscriber can modify the field's value on the Provider side, enabling dynamic configuration or state changes.

c) SOME/IP On wire format

A SOME/IP on-Wire Format message consists of two main components: a header and a payload as presented in Figure 2.3. The header includes key information about the message, such as the service ID, method ID, length, client ID, session ID, protocol version, interface version, and message type. The payload contains the actual data being transmitted.

- **Service ID:** A unique identifier assigned to each service.
- **Method ID:** An identifier that corresponds to the specific method being invoked on the service. Method IDs are divided into two ranges: 0-32767 for methods and 32768-65535 for events.
- **Length:** The size of the payload, measured in bytes.
- **Client ID:** A unique identifier assigned to the calling client within the ECU.
- **Session ID:** An identifier used for session management; it must be incremented with each new call.
- **Protocol Version:** Indicates the version of the SOME/IP protocol in use.
- **Interface Version:** Represents the major version number of the service interface in use.
- **Message Type:** Defines the category of the message being exchanged. The supported types include:
  - **REQUEST (0x00):** A message sent to request data from a service.

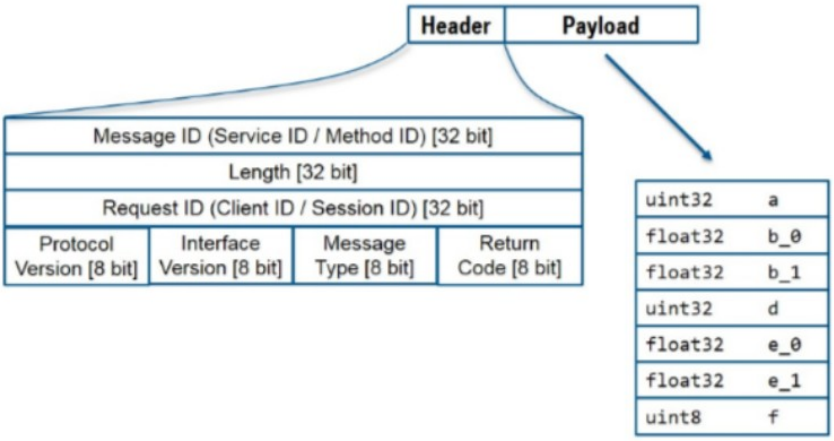

Figure 2.3: Message Format[14] .

  - **REQUEST NO RETURN (0x01):** A data request that does not expect a response.
  - **NOTIFICATION (0x02):** Indicates that a particular event has occurred.
  - **RESPONSE (0x80):** A reply sent in response to a previous request.

- **Return Code:** Communicates the result of the operation. The primary return codes are:
  - **E_OK (0x00):** The operation completed successfully.
  - **E_NOT_OK (0x01):** The operation failed.
  - **E_WRONG_INTERFACE_VERSION (0x08):** The client's interface version does not match the server's supported version.
  - **E_MALFORMED_MESSAGE (0x09):** The received message is invalid or improperly- formatted.

2.3.3 CAN protocol

CAN is a serial, multi-master, message-based protocol developed for in-vehicle communication. It enables multiple ECUs within a vehicle to exchange information directly, without the need for a central host computer [15] Its primary purpose is to provide a reliable and cost-effective means for sharing information between different electronic systems, enabling coordinated functionalities and reducing the complexity and weight associated with point-to-point wiring.

CAN's key characteristics include its resilience to electromagnetic interference, its ability to prioritize messages, and its inherent error detection and signaling mechanisms, making it suitable for safety-critical applications.

a) Architecture and Communication Mechanism

CAN uses a message-based communication paradigm. Instead of transmitting source and destination addresses, each message is identified by a unique message identifier (ID). This ID not only defines the content of the message but also determines its priority on the bus,lower numerical IDs have higher priority.

Each CAN message includes a header, which contains important fields such as the message ID, control bits, and a length field (which specifies the size of the data payload). The payload carries the actual data, followed by error-checking and acknowledgment fields . There are two main types of CAN protocols as shown in Figure 2.4:

1. **Classical (Standard) CAN:** This protocol supports a maximum data payload of 8 bytes. The frame structure consists of an 11-bit Identifier, a 6-bit Control field, a 4-bit Length field, a Data field (1–8 bytes), a 15-bit CRC, an Acknowledgment (ACK), and a 7-bit End of Frame (EOF) [16].
2. **CAN FD (Flexible Data-Rate):** An extension of the Classical CAN protocol, CAN FD increases the maximum payload size to 64 bytes and supports higher bit rates during the data phase. Its frame format includes an expanded 22-bit CRC while maintaining a similar overall structure with enhanced capabilities. This makes CAN FD ideal for applications requiring high-speed and high-volume data transmission [17].

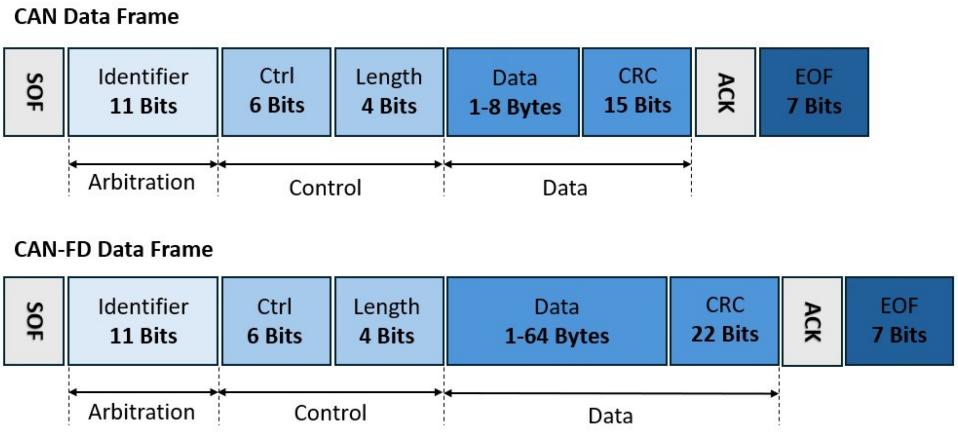

Figure 2.4: CAN and CAN FD Data Frame Formats [18] .

b) Input and Output Frames

From the perspective of a specific ECU on the CAN bus, messages can be categorized as:

- Output Frames

These are the CAN Data Frames that a particular ECU transmits onto the CAN bus. The data contained within these frames represents information generated or processed by that ECU to be shared with other ECUs.

For example: An engine control unit might transmit an output frame containing the current engine speed and temperature. The message ID is typically associated with the type of data being transmitted.

-Input Frames

These are the CAN Data Frames that a particular ECU receives from the CAN bus. The data within these frames contains information originating from other ECUs needed for its operation.

Example: A brake control unit might receive input frames with wheel speed information from another ECU to determine if anti-lock braking is necessary.

Note: A single CAN message frame can act as an output frame for the transmitting ECU and simultaneously serve as an input frame for multiple receiving ECUs configured to listen for that specific message ID.

c) Channels (Logical)

While the physical CAN bus consists of two wires, the concept of *channels* in automotive software often refers to logical groupings within the data. For example:

- A CAN message carrying multiple sensor readings might treat each reading as a logical channel.
- Functional areas (e.g., powertrain, chassis, body electronics) may be grouped into logical channels, even if on the same physical bus.
- Higher-layer protocols may define specific channels for various data types or service requests.

These channels are typically defined in communication matrices and represent specific data signals or signal groups.

d) Clusters (Physical and Logical)

The term **cluster** in CAN context can refer to both physical and logical groupings:

-Physical Cluster

- A single physical CAN bus segment where multiple ECUs are connected to the same two wires.
- All ECUs within this cluster can communicate directly, subject to arbitration and filtering.
- A vehicle may contain multiple physical clusters, e.g., a high-speed powertrain CAN and a low-speed body CAN, interconnected by gateway ECUs.

- Logical Cluster

- A group of ECUs that communicate for a specific function or subsystem.
- They may not reside on the same physical bus.

Logical clusters help manage communication dependencies and network design, often represented in configuration tools and software architecture.

2.4 Overview of DB Files in Vehicle Network Architecture

This section explores the architecture and applications of DB files in vehicle networks, detailing their role in automotive testing, validation, and ECU test case generation.

2.4.1 Database File Architecture

DB file in the automotive context,often structured using formats such as SQL,is a critical component containing tables that represent ECUs, network endpoints, services (for Ethernet), or communication frames (for CAN/CAN-FD). It includes the essential configuration data required for ECUs to function correctly within the vehicle's communication network. This data defines how ECUs exchange information by specifying message formats, signal definitions, and communication parameters, making the DB file a key resource for network configuration, diagnostics, and validation.

As shown in the figure 2.5, the DB file is organized into three clusters: Ethernet, CAN,CAN -FD, and FlexRay. This project focuses on the Ethernet and CAN clusters.

In the Ethernet cluster, ECUs can act as service providers or consumers. Each service may consist of methods, events, and fields, as described in earlier sections.

In the CAN cluster, communication is modeled using input or output frames. Each frame contains a Protocol Data Unit (PDU), which aggregates all signals transmitted or received in that frame.

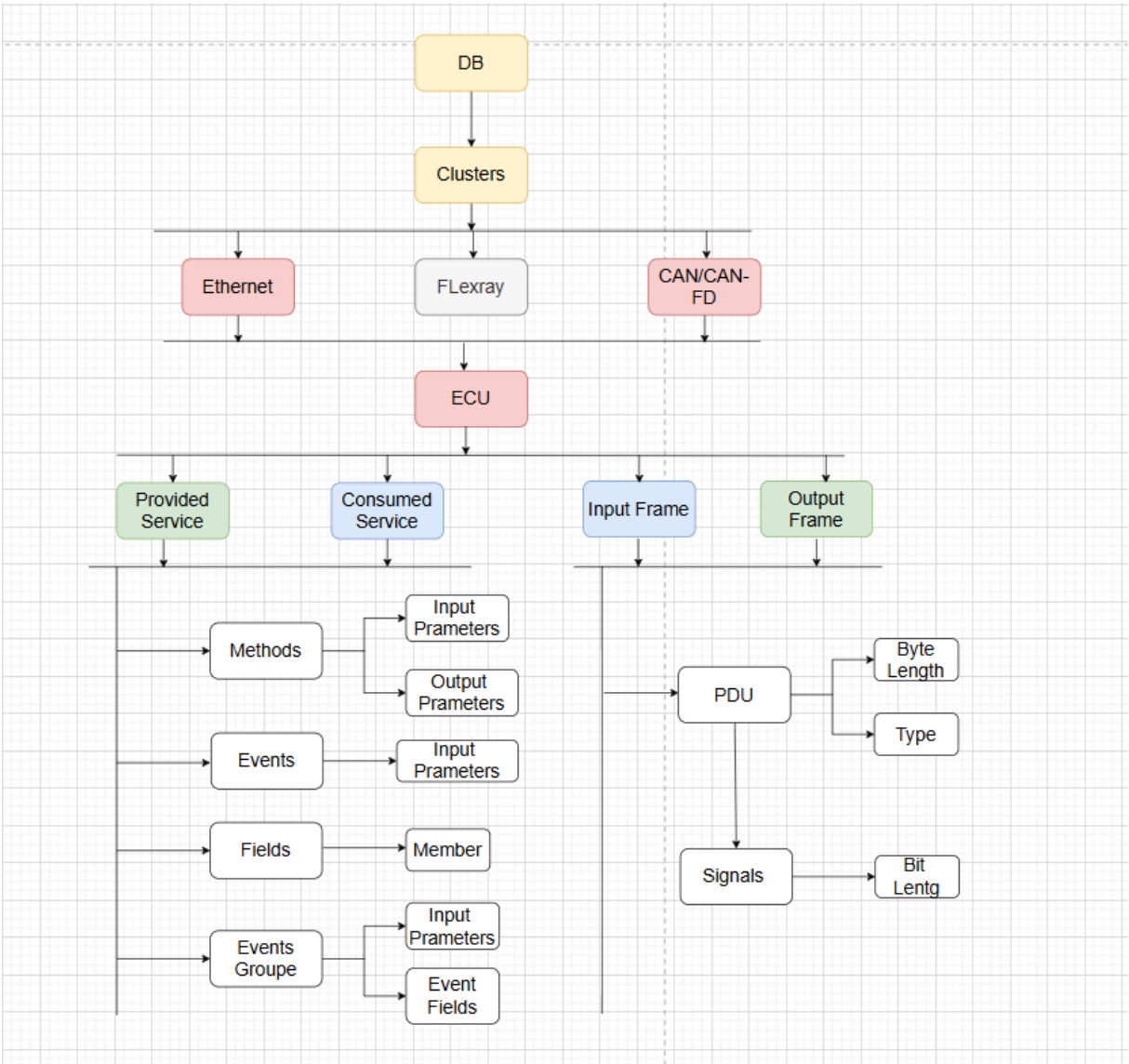

Figure 2.5: Database file architecture.

2.4.2 Role of DB Files in Automotive Testing and Validation

In the automotive industry, DB files play a vital role in the testing and validation of ECUs and communication networks.These files typically define the structure of communication messages exchanged over in-vehicle networks such as CAN, Ethernet. A DB file contains detailed specifications of signal names, message IDs, data lengths, scaling factors, and timing parameters. By using DB files, testing tools and simulation platforms can interpret, inject, and monitor network messages with accuracy, ensuring that each system behaves according to design

specifications. This is essential for validating functions such as engine control, braking systems, and ADAS, thereby contributing to the overall safety and reliability of the vehicle.

2.4.3 Test Case Generation from DB Files in ECU Testing

One practical example of using a DB file in the automotive testing process is in the generation of test cases for ECUs that either provide or consume specific messages on the vehicle network. The DB file contains detailed configuration data ,including message identifiers, signal definitions, and timing parameters,which can be used to automatically generate test scenarios.

For instance, when testing an ECU expected to transmit or receive certain CAN frames or Ethernet service messages, the DB file serves as a reference to ensure that the message structure and content are correct. Automotive validation platforms,such as ANDI ,leverage the DB file to simulate the network environment, verify communication behavior, and detect deviations from expected message formats or timing constraints. This approach significantly enhances the accuracy and efficiency of ECU validation.

In the context of our project, we aim to generate a script to verify that the configuration ensures messages are properly sent and received by retrieving the necessary data from the database for both the Ethernet and CAN protocols.

2.5 Natural Language to SQL Chatbot

To facilitate the work of the tester in understanding the entire network architecture, we suggest building a chatbot that can comprehend the tester's questions and generate corresponding SQL queries. This chatbot serves as an intelligent interface between the tester and the underlying database, simplifying data access and reducing dependency on technical SQL knowledge.

The chatbot component of the system leverages a fine-tuned language model specifically designed to translate user-provided natural language questions into executable SQL queries. This enables non-technical users to interact with the underlying database without needing to write or understand SQL.

The model was trained on domain-specific question-query pairs to improve its ability to interpret the schema, field names, and common query intents specific to the application. By applying transfer learning, a pre-trained large language model was adapted to the task, significantly reducing training time while preserving general language understanding and improving accuracy in generating SQL for the custom schema.

To handle variations in phrasing and vocabulary, fine-tuning also included examples with paraphrased inputs, synonyms, and natural variations in how users may ask questions.

2.5.1 Defog SQLCoder

To enable this functionality, we adopted **Defog SQLCoder**. It is an advanced AI model capable of translating natural language queries into SQL statements [19] It is trained on a massive dataset of SQL queries and their corresponding natural language descriptions. This allows the SQLCoder to understand the meaning of a natural language description and to generate the equivalent SQL query.It is built on the **LLaMA** (Large Language Model Meta AI) architecture,a suite of foundational language models ranging from 7B to 65B parameters,trained on publicly available datasets and optimized for efficiency and performance across diverse natural language processing tasks [20]. By leveraging the capabilities of LLaMA and pretraining on large-scale datasets, SQLCoder particularly
well-suited for database-oriented reasoning and structured query generation, making it an ideal choice for this project.

2.5.2 Advantages of SQLCoder

SQLCoder offers several compelling features that make it a excellent choice for text-to-SQL tasks[21]:

- **High Accuracy:** SQLCoder demonstrates superior performance compared to many general -purpose LLMs (LargeLanguage Model)on standard text-to-SQL benchmarks.

- **Schema Awareness:** The model is designed to incorporate structured schema inputs, enhancing its understanding of database-specific context and improving query generation.
- **Open-Source and Extensible:** As an open-source model, SQLCoder supports cost-effective- deployment and offers flexibility for custom fine-tuning based on specific use cases.
- **Efficiency:** SQLCoder-7B offers an optimal trade-off between performance and comput -ational- efficiency, making it well-suited for environments with limited resources.

The bar chart in Figure 2.6 visualizes the percentage of correctly generated SQL queries on novel schemas not seen during training

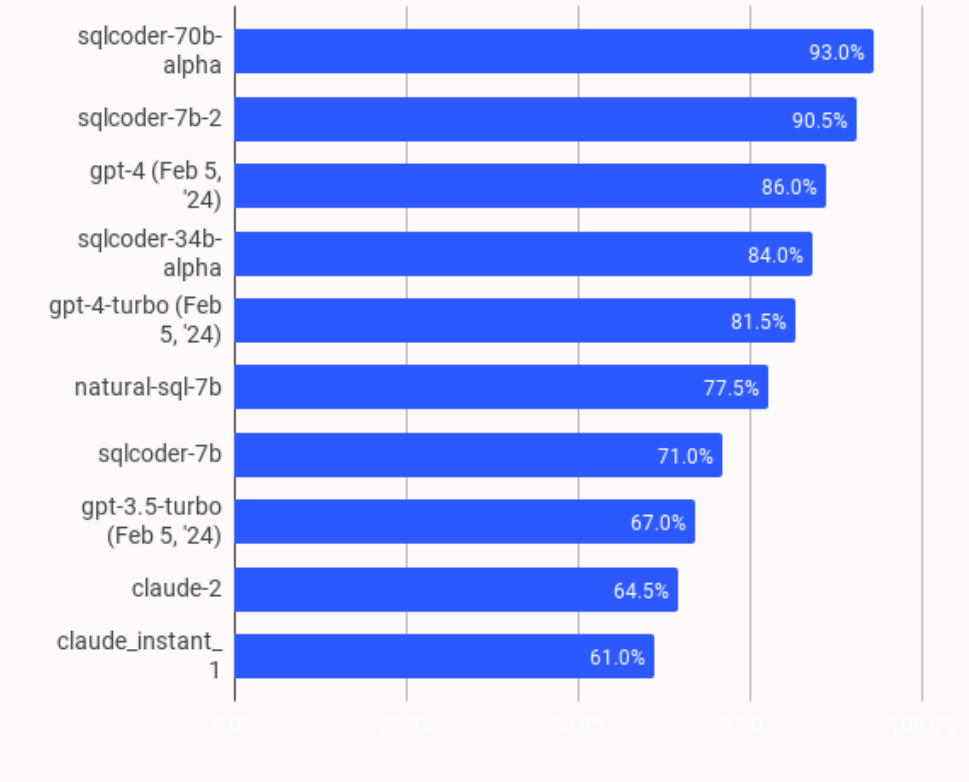

Figure 2.6: Performance comparison between SQLCoder and other LLMs [22].

Based on the benchmark results, sqlcoder-7b-2 offers a compelling balance between performance- and efficiency. With an accuracy of 90.5 percent on novel SQL schema generation tasks, it performs nearly as well as the top-performing sqlcoder-70b-alpha (93.0 percent) while being significantly smaller and more hardware-friendly. This makes sqlcoder-7b-2 the best practical choice for environments with limited compute resources, which makes this model the

best choice to be fine-tuned to generate SQL code related to the DB, as it combines strong out-of-the-box performance with a manageable computational footprint ideal for customization

2.6 CONCLUSION

In conclusion, this chapter has provided a foundational overview of vehicle network architecture. We began by establishing the context of the automotive industry and emphasizing the central role of ECUs, along with the importance of standardized frameworks such as AUTOSAR. We then explored key technical concepts, comparing different automotive network architectures, including Ethernet SOME/IP and the CAN bus system. Additionally, we discussed the critical role of DB files in understanding, testing, and validating vehicle network communication, particularly in generating ECU test cases. Finally, we introduced the proposed system model and justified the integration of Defog SQL Coder as an intelligent chatbot to enhance interaction. With the network fundamentals and system model now established, the next chapter will focus on the detailed analysis and design of the project.

Chapter

3 Project Analysis and Design

Contents

3.1 INTRODUCTION

Chapter 3 establishes a structured foundation for Project Analysis and Design. This chapter will guide you through the essential initial phases of any project, focusing on understanding the core needs and translating them into a well-defined design. We begin with a critical needs Analysis, which involves identifying all relevant stakeholders and meticulously specifying both the functional requirements that the project must fulfill and the non-functional requirements that define the quality attributes and constraints of the project. Following this crucial analysis, the chapter delves into UML Design, a powerful visual modeling language. We will explore the creation and interpretation of key UML diagrams, including the Global Use Case Diagram for understanding system scope, textual descriptions to elaborate on use cases, the Class Diagram for representing the system's structure, and the Sequence Diagram for illustrating dynamic interactions between objects.

3.2 Needs Analysis

Each proposed solution must address a defined set of functional and non-functional requirements. This section begins with the identification of the relevant stakeholders, followed by a detailed presentation of the identified needs.

3.2.1 Stakeholder Identification

The users of our application are:

- **User (Tester):** The tester is an individual who interacts with the interface to fulfill specific needs. This role involves representing data and generating test cases through the application.
- **Administrator:** The administrator is responsible for managing access to the application for testers, ensuring appropriate user permissions and system security.

3.2.2 Functional Requirements Specification

Functional requirements describe the actions and responses the system must perform in reaction to a request from a primary stakeholder. The solution must provide the following capabilities to its users:

- **Display Data:** The application allows the user to upload a file and automatically generate diagrams and associated grids to facilitate understanding of the client's database.
- **Generate Code:** The user can generate source code based on each of the represented diagrams.
- **User and Admin Dashboards:** The system provides separate dashboards for users and administrators, each with role-specific functionalities and interfaces.

3.2.3 Non-Functional Requirements Specification

This section outlines the requirements that ensure the delivery of a reliable product, capable of meeting user expectations while minimizing risks of failure or malfunction. The non-functional requirements for our application are as follows:

- **Performance:** The application must operate efficiently, effectively addressing user needs with minimal delay.
- **Response Time:** The system must respond promptly to user actions within an acceptable timeframe.
- **Scalability:** The codebase should be clear and modular to facilitate future enhancements or expansions.
- **Availability:** The system must remain consistently accessible.
- **Usability:** The application should provide a user-friendly and intuitive interface.

3.3 Global Conception

In this section, we present the use case diagram along with its corresponding textual description.

3.3.1 Global Use Case Diagram

In this part, we present the global use case diagrams for the Tester and Administrator actors. These diagrams serve to capture and describe the requirements of the system's stakeholders. They provide a user-centered representation of the system's functionalities and are used to model one of the static aspects of the system. Figure 3.1 illustrates the use case diagram for the Tester.

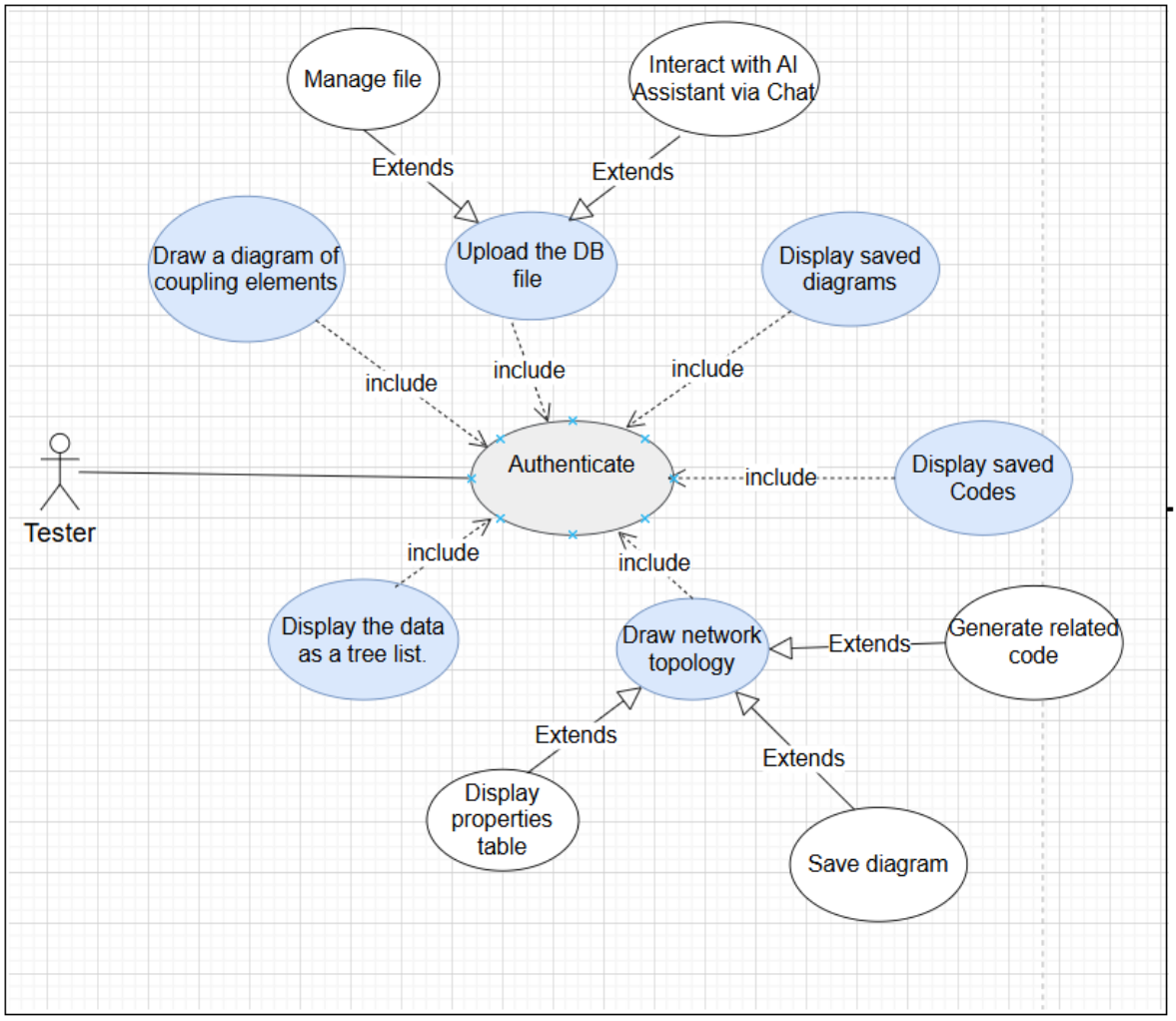

Figure 3.1: Use case diagram "User".

Figure 3.2 presents the use case diagram for the Administrator.

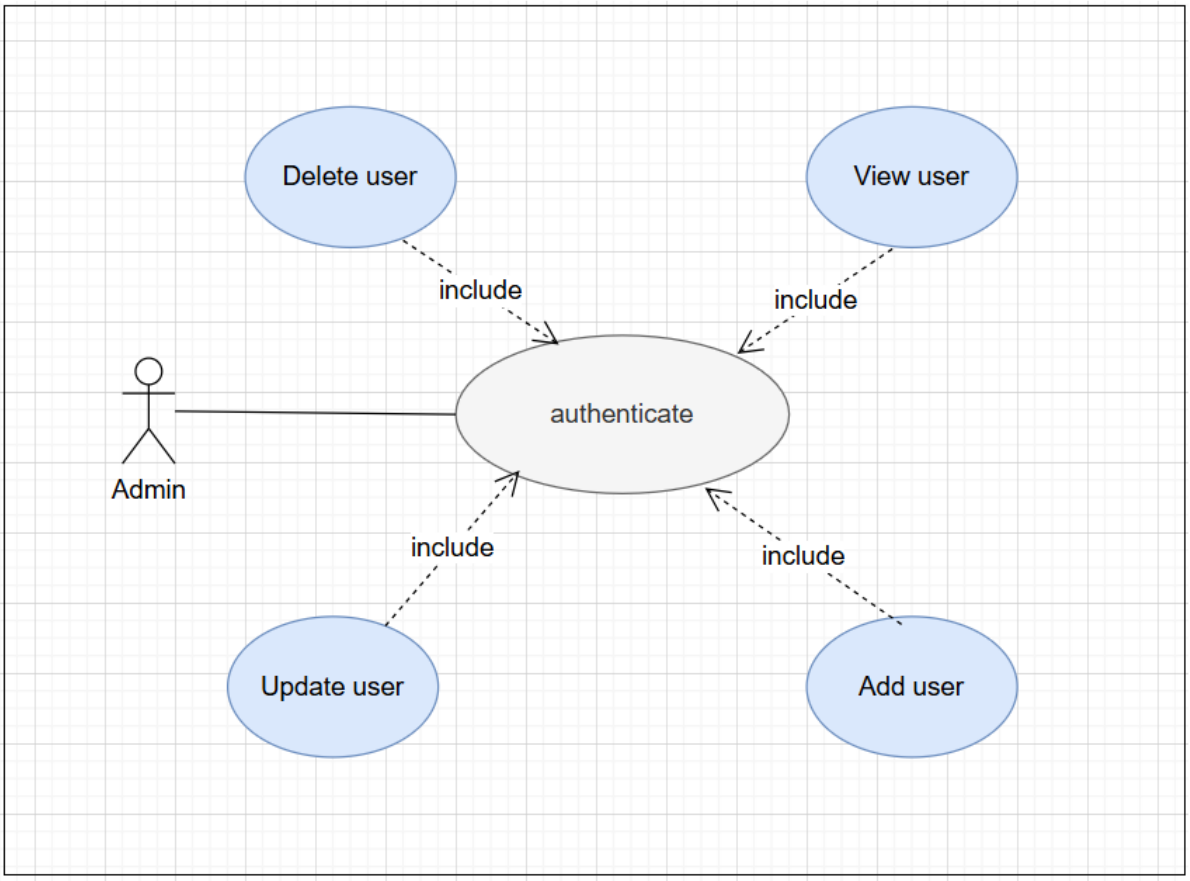

Figure 3.2: use case diagram "Admin".

3.3.2 Textual Description

The textual description of a use case helps clarify how the functionality unfolds and outlines the sequence of actions to be performed. These descriptions can also assist in identifying additional use cases that may be incorporated into the system.

3.3.2.1 Textual description of the scenario "Upload File "

Table 3.1 provides the textual description of the file upload process.

Table 3.1: Textual description of the scenario "Upload File done by the tester"

Title	**Upload DB File**
Actor	the tester
Summary	The application allows the tester to upload a DB file containing Ethernet or CAN data.
Description of sequences	
Preconditions	The tester opens the application, logs in, and must be working at KPIT.
Post conditions	The applications extract data from the DB and save it on the server DB.
Nominal Scenario	
• The user chooses the file. • The user selects the file type.	
Alternative Scenario	
• Failed due to an internal application-level error (for example, failed to connect to the database).	

3.3.2.2 Textual description of the scenario "Load The Tree List"

Table 3.2 describes the textual description of the process of loading the tree list.

Table 3.2: Textual description of the scenario "Load the Tree List done by the tester "

Title	**Display the data as tree list**
Actor	The Tester

Summary	The application allows the tester to view ECU-provided consumed services and input/output frames in a tree list format.
Description of sequences	
Preconditions	the tester has the access to this file
Post conditions	The application will associate each Ecu with consumed and provided service or frame as Tree View.
Nominal Scenario	
• The user clicks on the view icon • The app renders the tree list.	
Alternative Scenario	
• Failed due to an internal application-level error (for example, the user does not have access).	

3.3.2.3 Textual description of the scenario "Draw Network Topology"

Table 3.3 presents the textual description of the process to drawn a network topology.

Table 3.3: Textual description of the scenario "Draw Network Topology done by the tester"

Title	**Draw network topology**
Actor	The tester
Summary	Enable the user to create diagrams of ECUs and their connections.
Description of sequences	

Preconditions	the tester load the tree list and select Ecu and service/frame.
Post conditions	Each ECU's consumed and provided services or frames will be associated and displayed as a diagram
Nominal Scenario	
• The user clicks on the view icon • The app renders the tree list. • the user select Ecu/service/frame	
Alternative Scenario	
• Failed due to an internal application-level error (for example, the user has not the access an error will be raised).	

3.3.2.4 Textual description of the scenario the scenario "Manage Users"

Table 3.4 presents the textual description of the process for managing tester accounts.

Table 3.4: Textual description of the scenario "Manage Users"

Title	**add user**
Actor	Admin
Summary	The application shows all added users.
Description of sequences	
Preconditions	The administrator logged in.
Post conditions	the tester account is created
Nominal Scenario	

• The user clicks on the add button • The app shows a form to fill with tester credentials. • the user click on the sumbit button
Alternative Scenario
• Failed due to an internal application-level error (for example, the input are not valid or the user is already created .)

3.4 Detailed conception

In this section, we present the overall class diagram that support the detailed design of the system. Then, we present the sequence diagrams which play a crucial role in modeling functional requirements, as they graphically represent the order and flow of interactions between various objects or components. These diagrams are particularly useful for illustrating the dynamic behavior of the system, especially in complex scenarios or use cases. In parallel, the class diagram provides a comprehensive view of the system's static structure by depicting the relationships between the different classes. Together, these models contribute to a clearer understanding of both the structural and behavioral aspects of the system

3.4.1 Class Diagram

The use of a class diagram allows us to visually and concisely represent the structure of the system under study, highlighting the key classes, their attributes, and their relationships. This approach enhances the clarity of the system's architecture for readers and strengthens the quality

of our analysis and design. Figure 3.3 illustrates the class diagram representing the structure of our project.

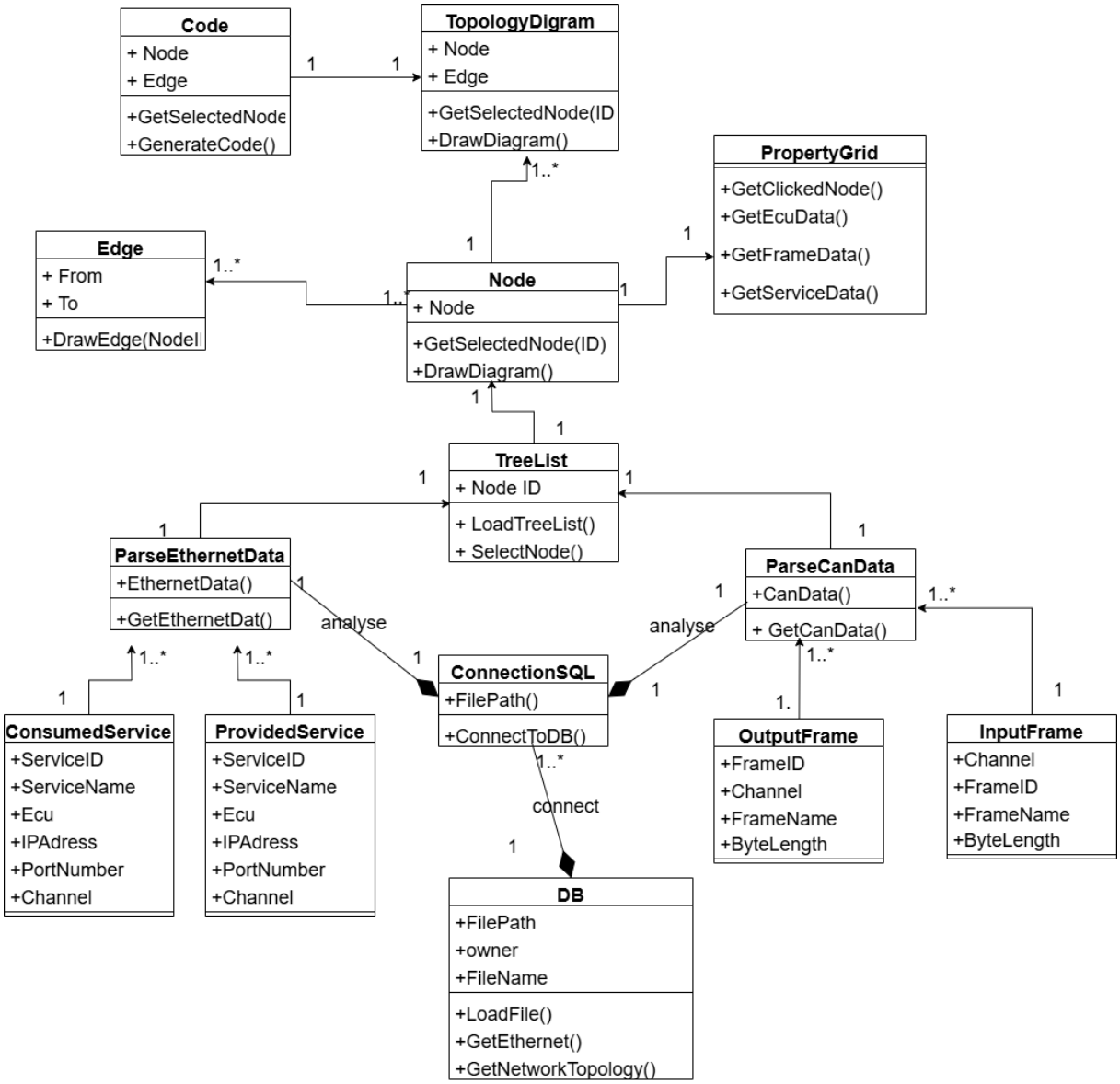

Figure 3.3: Class Diagram.

3.4.2 Sequence Diagram for "User Authentication"

Figure 3.4 illustrates the sequence of interactions that occur during the user authentication process. It details how a user initiates a login request, how the system verifies the provided credentials, and how access is granted or denied based on the outcome of the verification process.

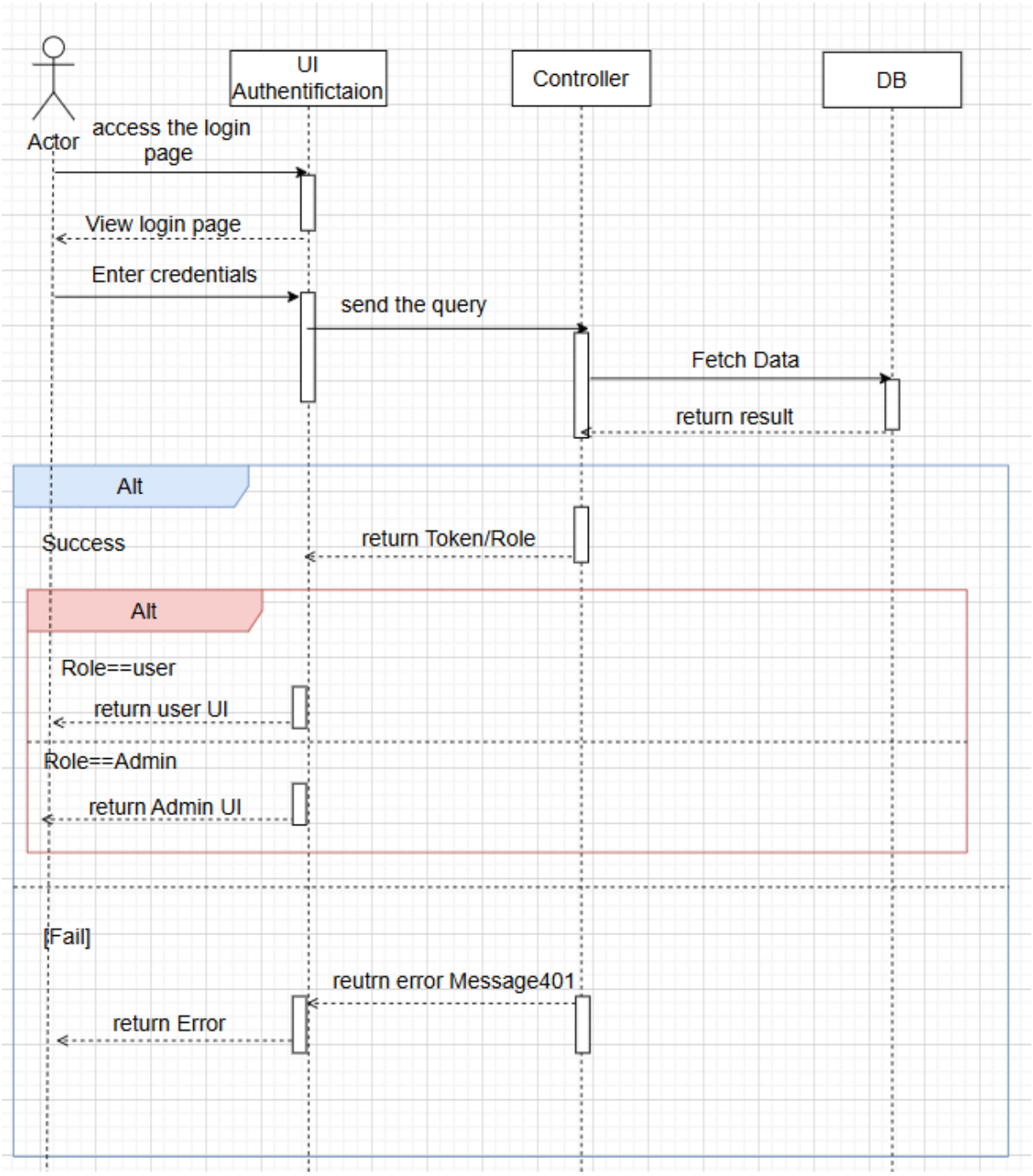

Figure 3.4: Sequence Diagram for "User Authentication".

3.4.3 Sequence Diagram for"Uploading File"

Figure 3.5 presents the sequence diagram for the file upload process. It demonstrates how a user selects a file, how the system handles the upload request, and the subsequent steps involved in storing the file and confirming a successful upload to the user.

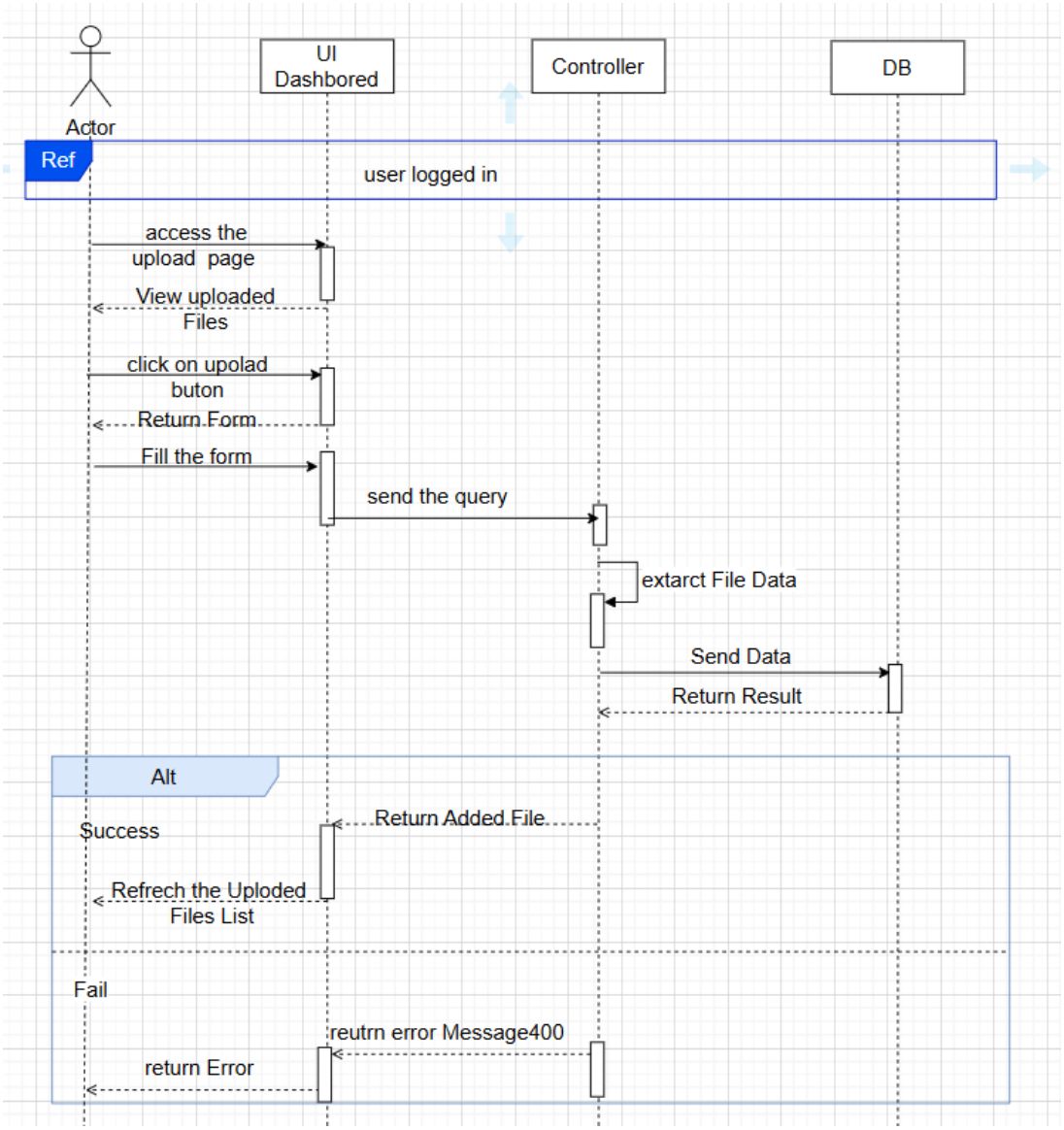

Figure 3.5: Sequence Diagram for "Uploading File".

3.4.4 Sequence Diagram for "Network Topology"

Figure 3.6 present the sequence diagram illustrating the process of network topology construction. It details the sequence of actions initiated by the user's selection of an item and the system's subsequent generation of the associated nodes and edges.

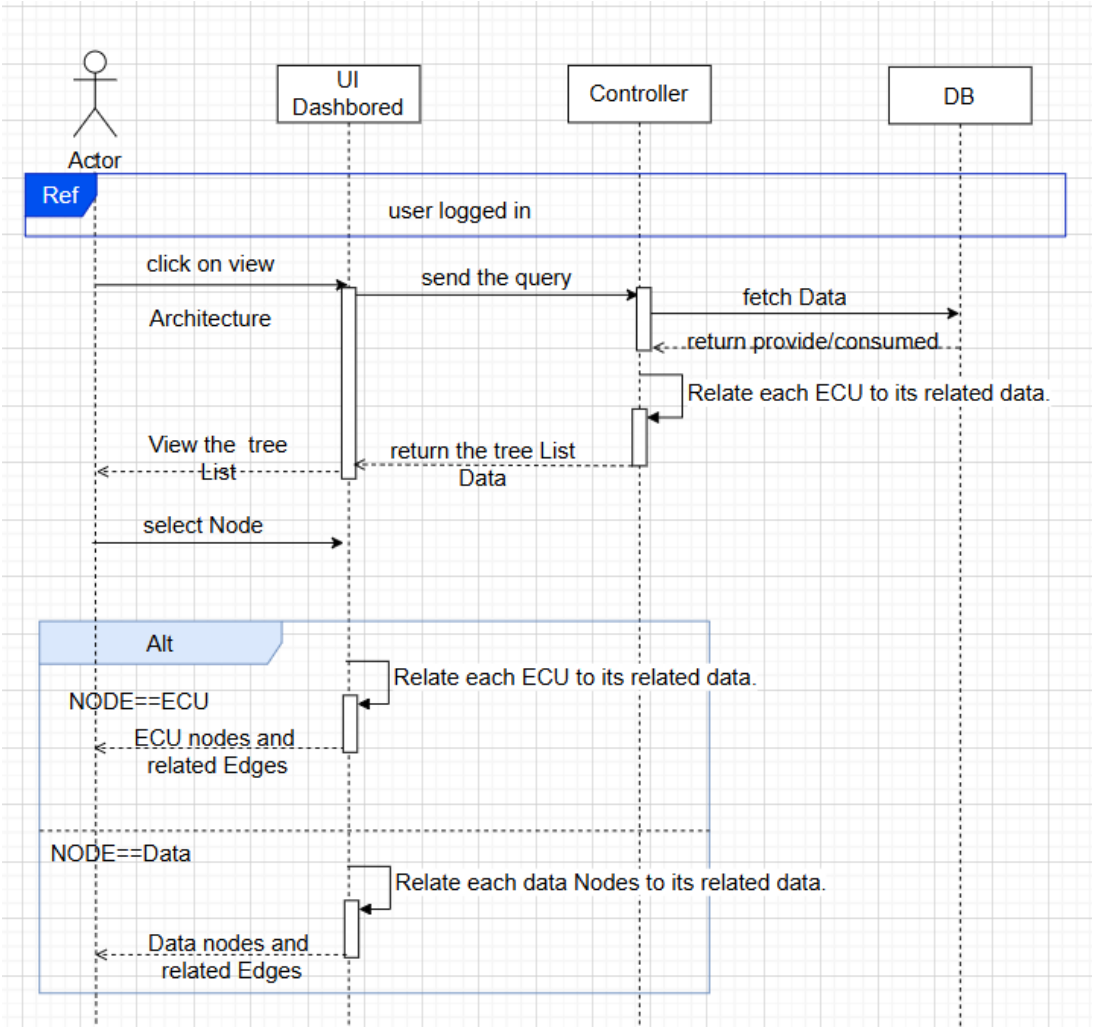

Figure 3.6: Sequence Diagram for "Network Topology".

3.4.5 Sequence Diagram for "Node Details"

Figure 3.7 presents the sequence diagram that illustrates the process by which a user retrieves detailed information about a node.

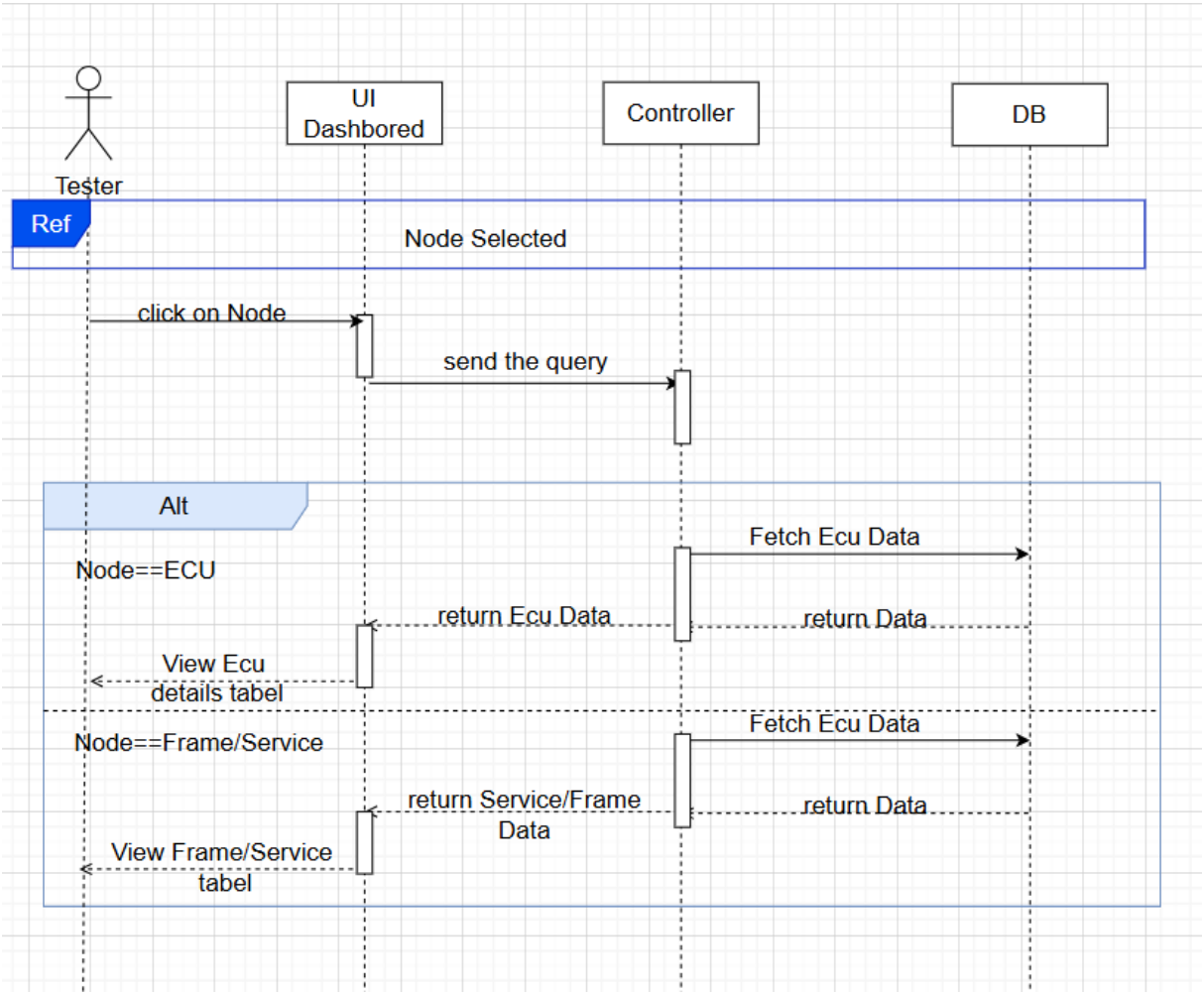

Figure 3.7: Sequence Diagram for "Node Details".

3.4.6 Sequence Diagram to "Generate Code"

Figure 3.8 presents the sequence diagram that illustrates the process in which the user can generate code from the selected diagram.

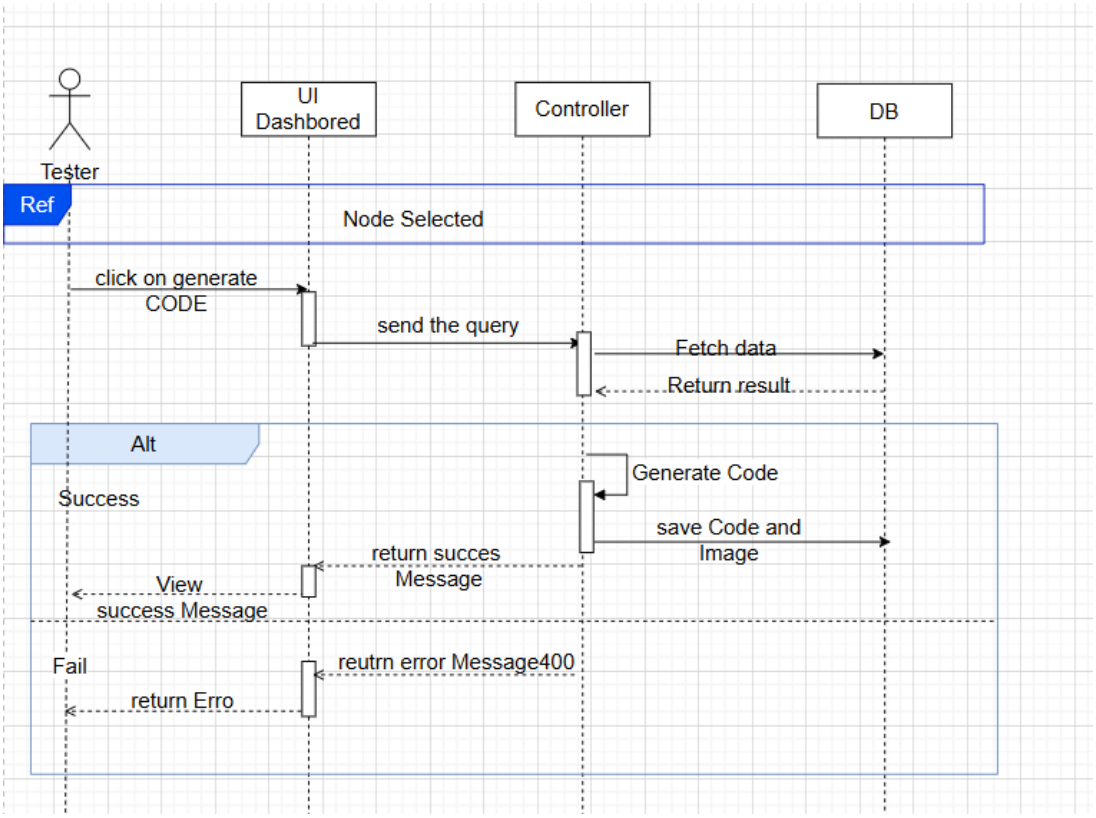

Figure 3.8: Sequence Diagram to "Generate Code".

3.4.7 Sequence Diagram "chat with sqlcoder"

Figure 3.9 presents the sequence diagram that illustrates the process in which the user can chat with sqlcoder to get the DATA he needs

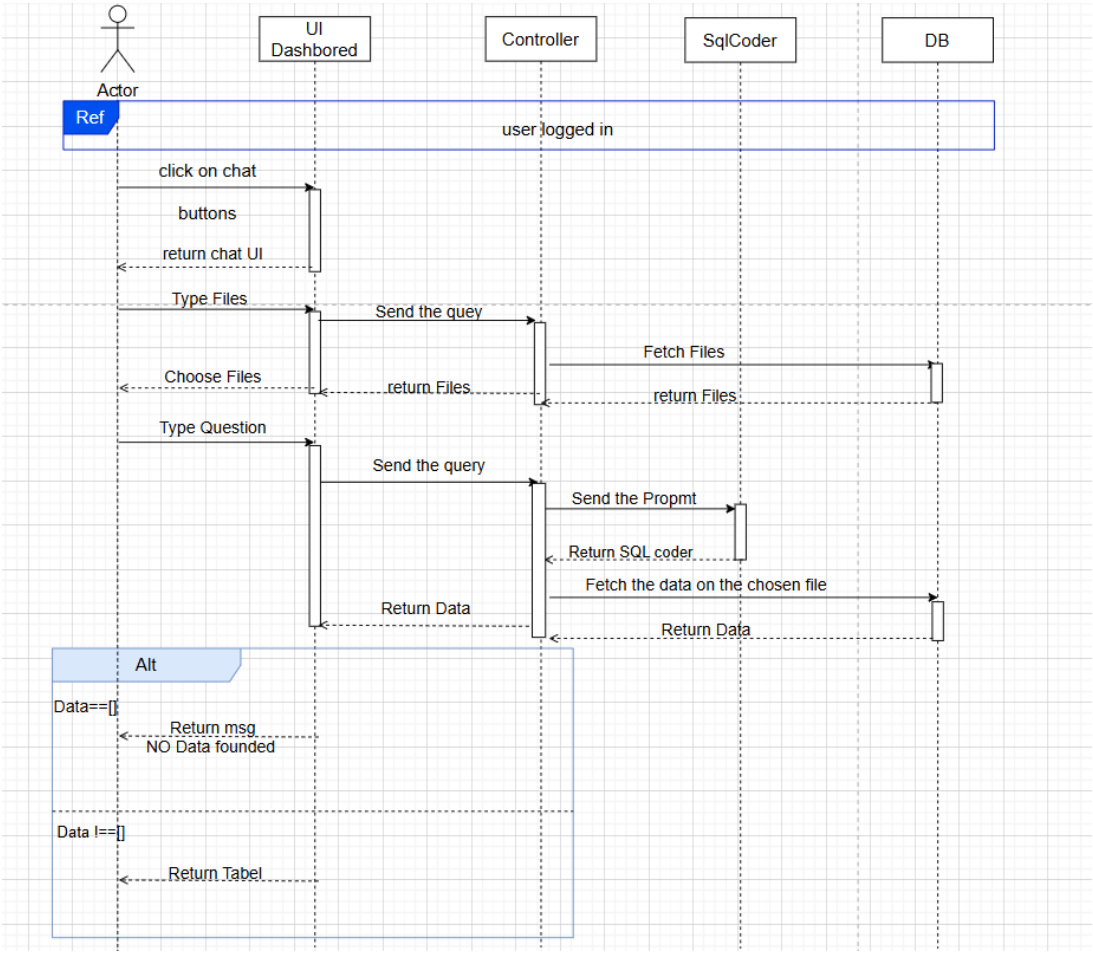

Figure 3.9: Sequence Diagram "Chat With Sqlcoder".

3.5 Conclusion

In conclusion, this chapter has provided a comprehensive overview of the initial phases of Project Analysis and Design. We emphasized the significance of a thorough needs Analysis, ensuring that project efforts are aligned with stakeholder expectations and that all essential functional and non-functional requirements are clearly defined. Furthermore, we explored the application of UML Design as a valuable methodology for visualizing and documenting the system's structure and behavior through various diagrams. Following the completion of the UML design, the project progresses to the realization phase, which will be discussed in detail in the next chapter.

Chapter

4 Realization

Contents

4.1 Introduction

This chapter presents the implementation details of the system introduced in the previous chapter. It describes the technical realization of the core components, including backend and frontend development, chatbot integration, and the overall system architecture. The focus is placed on the practical aspects of development, showing how the conceptual design was transformed into a functional application. The chapter begins with an overview of the system's architecture and the main frameworks used. It then addresses the integration of the chatbot, specifically emphasizing the use of NLP for automated SQL query generation. The final section is dedicated to the user interface and user experience (UI/UX), detailing the layout and functionalities of key system views such as the landing page, dashboard and code generation

4.2 Overview of the System

The project is structured as a full-stack web application with a clear separation of concerns between frontend and backend components. The architecture allows for efficient development, scalability and long-term maintainability.

4.2.1 Backend – Django REST Framework

The application's backend is developed using Django, a high-level Python web framework known for enabling rapid development and promoting clean, pragmatic design [23].To expose the underlying data and business logic to the frontend, Django REST Framework (DRF) is employed. DRF offers a robust and flexible toolkit for developing Web APIs, simplifying key aspects such as serialization, authentication, and routing. This results in a scalable and maintainable API architecture.

API endpoints deliver structured data in JavaScript Object Notation (JSON) format, enabling smooth communication with the frontend. Django REST Framework was chosen for its maturity,

active community support, and seamless integration with Django's built-in Object-Relational Mapper (ORM).

A core strength of DRF lies in its layered architecture, where each component is responsible for a specific role in the data flow between the client and the server. The table 4.1 outlines the main components of DRF utilized in the project:

Table 4.1: Core components of Django REST Framework and their roles

DRF Component	Responsibility
Model	Handles data operations and communicates with the database.
Serializer	Transforms data between model instances and JSON representations (serialization and deserialization).
ViewSet / APIView	Processes API requests, communicates with models (often through serializers), applies business logic, and returns appropriate responses.

4.2.2 Frontend – React and Material UI

The frontend of the application is built using React, a strong JavaScript library designed for creating user interfaces, especially in single-page applications [24]. React facilitates seamless interaction with the backend via asynchronous Hypertext Transfer Protocol (HTTP) requests, typically through libraries such as Axios.

To enhance the visual design and consistency of the user interface, the project integrates Material UI (MUI), a widely adopted React component library that implements Google's Material Design system. MUI offers a rich set of pre-built, customizable UI components, enabling rapid development of clean and professional interfaces.

The combination of React and MUI ensures a responsive and intuitive user experience, with efficient rendering and modularity that support scalability and maintainability across the application.

4.2.3 Full System Architecture

The system adopts a client-server architecture, separating concerns between the frontend and backend layers. The backend, developed using DRF, serves as the core provider of data and business logic. It exposes RESTful API endpoints that deliver and receive structured data in JSON format.

The frontend, built with React, acts as the client-side interface, providing a dynamic and interactive user experience. It communicates with the backend asynchronously through HTTP requests, primarily using the Axios library. Axios simplifies the process of sending requests and handling responses, making it straightforward to fetch data, submit forms or trigger actions such as user authentication.

When the frontend needs to retrieve or send data, Axios sends a request to the appropriate DRF endpoint (e.g., ‘GET‘, ‘POST‘, ‘PUT‘ or ‘DELETE‘).The DRF view handles the request, often by interacting with the database through Django models and converting data using serializers-. It then returns a structured JSON response to the frontend. React consumes this data and updates the user interface accordingly.

This interaction pattern ensures a decoupled, scalable, and maintainable architecture, with clearly defined responsibilities:

- **DRF (Backend)**: Handles data processing, validation and persistence.
- **Axios (Communication Layer)**: Manages HTTP requests/responses between the frontend and backend.
- **React (Frontend)**: Renders the user interface and reacts to data changes in real time.

This modular design facilitates efficient development and allows each layer to evolve independently while maintaining seamless integration.

4.3 Chatbot Integration

A key feature of the application is a chatbot that allows users to interact with the database using natural language. Acting as an intermediary layer, this component interprets user queries, translates them into corresponding SQL statements, sends them to the backend API, and displays the results in a clear and readable format.

To ensure that the chatbot can generate accurate queries aligned with the specific structure of the application's database, we fine-tuned the SQLCoder-7B-2 model, an open-source language model optimized for text-to-SQL translation.

The fine-tuning was performed using the Low-Rank Adaptation (LoRA) method, a computationally efficient technique that enables the injection of task-specific knowledge into a large model without retraining all of its parameters. Only a small set of low-rank matrices is updated, which significantly reduces training costs while preserving the model's ability to understand natural language.

The complete process of integrating the chatbot into a user-friendly interface required going through the following sequential stages. Each step played a critical role in ensuring the chatbot could accurately interpret user input, generate valid SQL queries, and present the results in an accessible format.

4.3.1 Data Preparation

To adapt the model to the vocabulary and data schema specific to the application, a custom dataset was created. It consists of natural language questions paired with their corresponding SQL queries. Each entry includes a prompt containing schema information, followed by explicit instructions and the user's question.

However, manually creating this dataset was not sufficient to ensure effective training. A semantic and structural data augmentation process was implemented, introducing variations in phrasing, the addition of conditional clauses (e.g., WHERE statements), query reformulations, and modifications of schema-specific elements such as ECU, service, or frame names.

As a result of this approach, the dataset was expanded to a total of 263 examples, providing the model with broader exposure to diverse query formulations.

4.3.2 Tokenizing the Dataset

Since we are using a LLM, and such models cannot directly interpret raw text, the data must first be tokenized.

Tokenization is the process of breaking down text into smaller units called tokens, which are typically words or sub-words in natural language processing. It is a fundamental step in various NLP tasks such as text processing, language modeling, and machine translation. This process involves dividing a string or text into a sequence of individual tokens [25].

The tokenizer converts human-readable text into a sequence of numerical identifiers (IDs) that the LLM can understand and manipulate internally. This transformation is essential for enabling the model to perform reasoning, pattern recognition, and query generation based on the textual input.

The tokenization process used in this project is illustrated in Figure 4.1. As shown in the

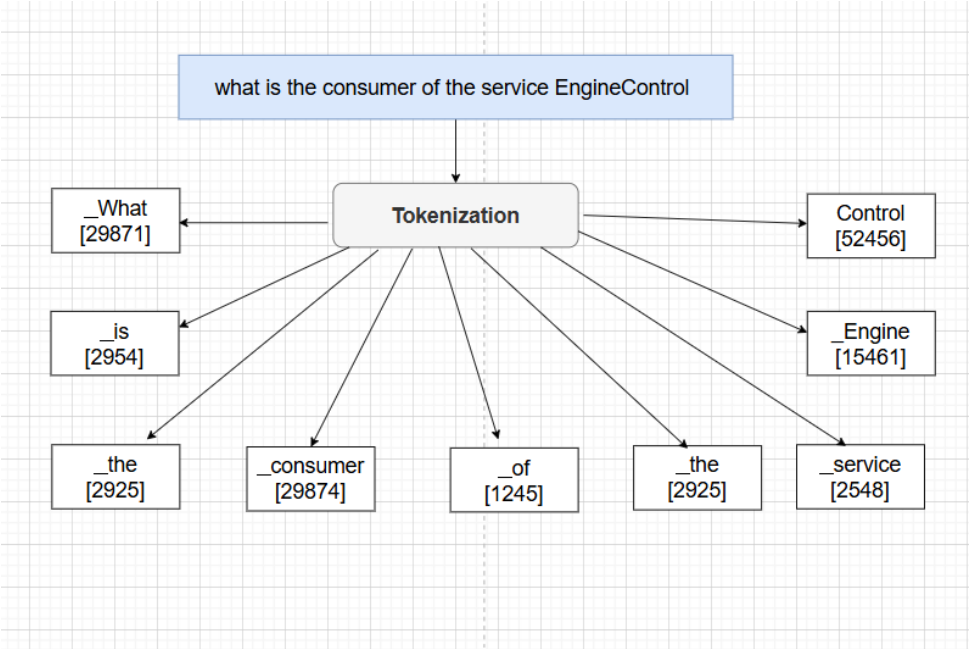

Figure 4.1: Tokenization process used for preparing model inputs.

figure, the pipeline begins with raw text segmentation, where the input query (eg "What is the

consumer of the service EngineControl") is split into contextual tokens. These tokens are then mapped to their corresponding IDs using a predefined vocabulary (e.g.,'-What', '-is', '-the', '-consumer', '-of', '-the', '-service', '-Engine', 'Control'). The figure highlights how special tokens (like [CLS] for classification tasks or [SEP] for separators) are inserted to structure the input. Finally, the sequence of IDs is passed through an embedding layer to generate dense vector representations, which preserve semantic relationships and positional context, critical for the LLM to generate accurate SQL queries.

4.3.3 LoRA Configuration

The fine-tuning process was configured to update only a small subset of the model's parameters using LoRA adapters. This technique introduces lightweight trainable layers into specific components of the model, thereby avoiding full parameter retraining and significantly reducing computational overhead.

In our setup, only two attention-related sublayers were fine-tuned: the query projection and value projection layers. These layers play a key role in guiding the model's attention and controlling the content it generates in SQL form.

- **Query Projection – "What am I looking for?"**

  This projection transforms the input hidden states into query vectors. These vectors define what each token is trying to attend to in the context of the sequence. In the case of SQL generation, the query projection shapes how the model seeks relevant schema elements, conditions, or values from the input.

- **Value Projection – "What information is available?"**

  This projection maps hidden states to value vectors, representing the actual content to be retrieved and aggregated during attention. In SQL generation, value projections help ensure that critical elements—such as table names, column values, or keywords—are preserved and correctly propagated through the model's layers during decoding.

4.3.4 Training

After tokenizing the data, the resulting dataset was split into two subsets: a training set and a test set. The model was trained over the course of three epochs.

Figure 4.2 below illustrates the evolution of the loss function during training. By the end of the final epoch, the training loss had decreased to 0.03, while the validation loss reached 0.008. These values indicate that the model was learning effectively and generalizing well to unseen data, as evidenced by the low validation loss. The small gap between training and validation losses further suggests that the model did not suffer from overfitting.

However, due to ressource-hardware limitations, it was not possible to compute the evolution of the loss on the test set across epochs, as was done for the training set. For the same reason, we were also unable to apply certain evaluation metrics commonly used in text-to-SQL tasks, such as Execution Accuracy. This constraint partially limits our ability to precisely assess the model's robustness on complex queries. Nevertheless, the validation results obtained remain encouraging, especially considering the restricted computational resources available during experimentation shows the training loss over the epochs. At the final epoch, the training loss was 0.03, and the validation loss was 0.008. This indicates that the model is learning effectively and generalizing well to unseen data, as evidenced by the low validation loss. The small gap between training and validation loss suggests that the model is not overfitting. A further qualitative evaluation,including an example of the chatbot's question-and-response interaction, will be presented in the Interface and User Experience section.

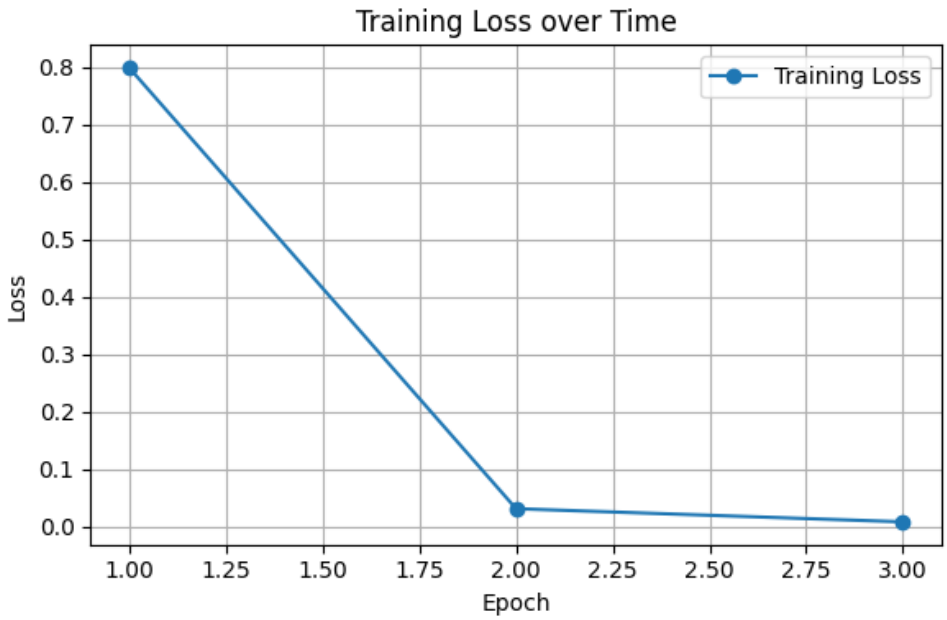

Figure 4.2: Training and validation loss over epochs.

4.3.5 Quantization

Quantization is a widely adopted technique aimed at reducing the computational complexity and memory footprint of machine learning models, particularly large LLMs. By compressing model parameters, quantization enables the deployment of these models on resource-constrained hard-ware- platforms such as personal laptops or small-scale servers, while maintaining an acceptable level of model performance .In this study, quantization was applied to the fine-tuned model, resulting in a variant referred to as, resulting in the variant denoted as `SQLCoder-Q5-K-M.gguf`.

The notation `Q5-K-M.gguf` specifies a particular quantization scheme implemented within the GGUF framework, the successor to the GGML library . GGML is a C library designed for efficient inference, LLMs to run on standard hardware—including CPUs and Apple Silicon—by leveraging techniques such as quantization to enhance speed and performance. It powers implem -entations- like `llama.cpp` [26]

The details of the Q5-K-M quantization scheme are as follows:

- **Q5**: Specifies that the model weights are quantized to 5-bit precision, significantly reducing memory usage and computational overhead compared to the standard 32-bit floating-point representation.
- **K-M**: Refers to a mixed-precision quantization strategy that divides model weights into blocks. This approach selectively applies different precision levels within those blocks, balancing the trade-off between compression and model accuracy.

4.4 Interface and User Experience

After presenting the core system components, including the backend and frontend technologies as well as the integration of the chatbot and its supporting architecture, we now shift our focus to the user interface and user experience aspects of the application.

While the backend ensures data processing and the chatbot enables intelligent query generation-, it is ultimately the interface that determines how efficiently and intuitively users interact with the system. This section describes the layout, design choices, and interactive elements implemented to provide a seamless user experience. The goal is to offer a responsive, accessible, and visually coherent environment that facilitates data exploration, test case generation, and overall usability.

4.4.1 Landing Page

The figure 4.3 presents the landing page of the web application. It provides an overview of the available services and features a navigation bar that allows users to access the core functionalities of the application.

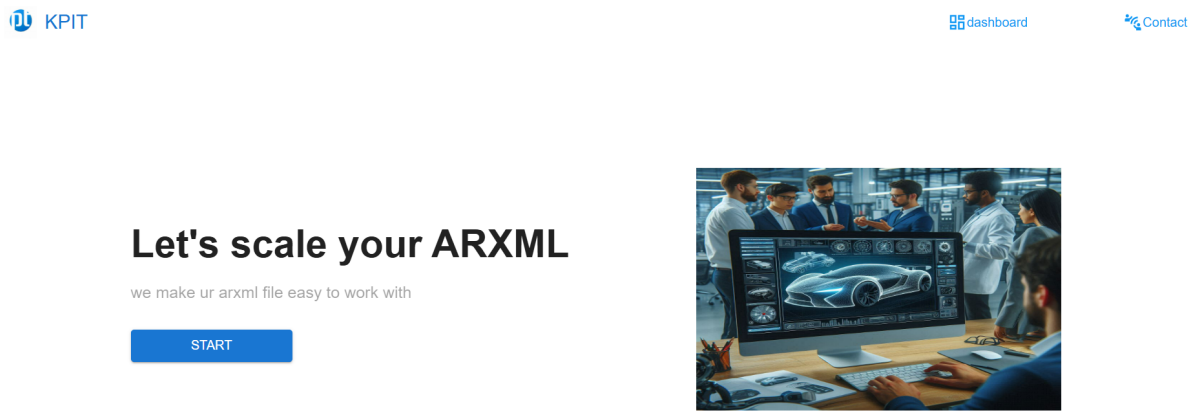

Figure 4.3: Landing Page

4.4.2 Login Page

The figure 4.4 shows the login page, which enables users to access the application by entering their email and password credentials.

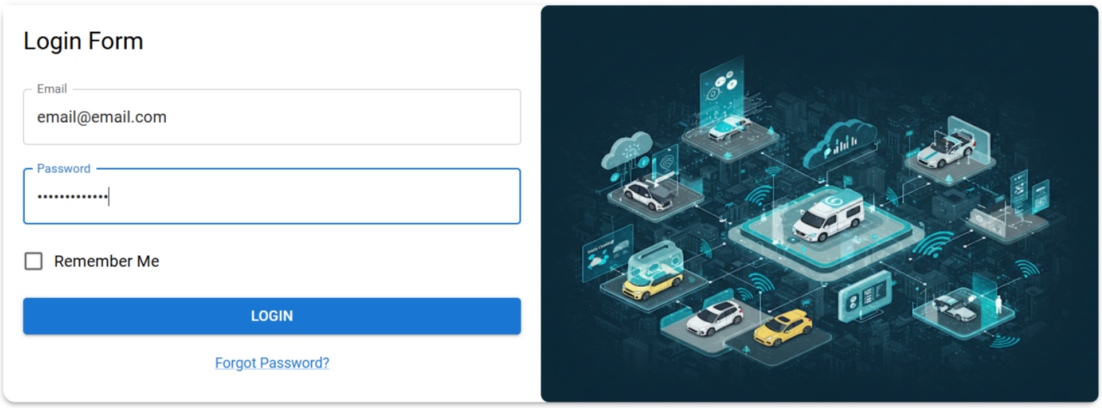

Figure 4.4: Login Page

4.4.3 Dashboard Page for User

The following figures 4.5 and 4.6 illustrate the dashboard page, where the tester can view their uploaded files presented in a tabular format. This interface allows the tester to examine the topology architecture of each file by selecting the corresponding button. Additionally, the tester has the capability to upload new files, edit existing ones, and share files with other testers.

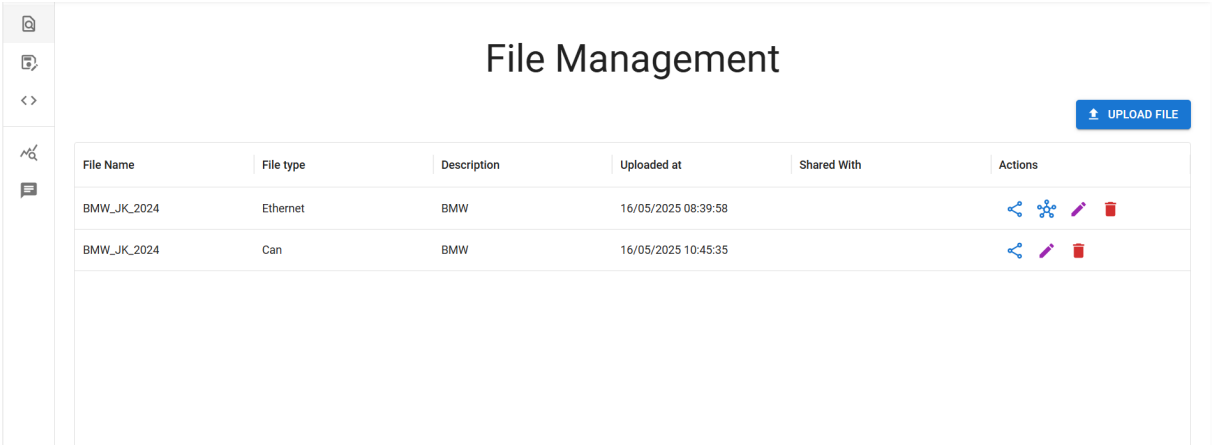

Figure 4.5: Dashboard page displaying the list of uploaded files.

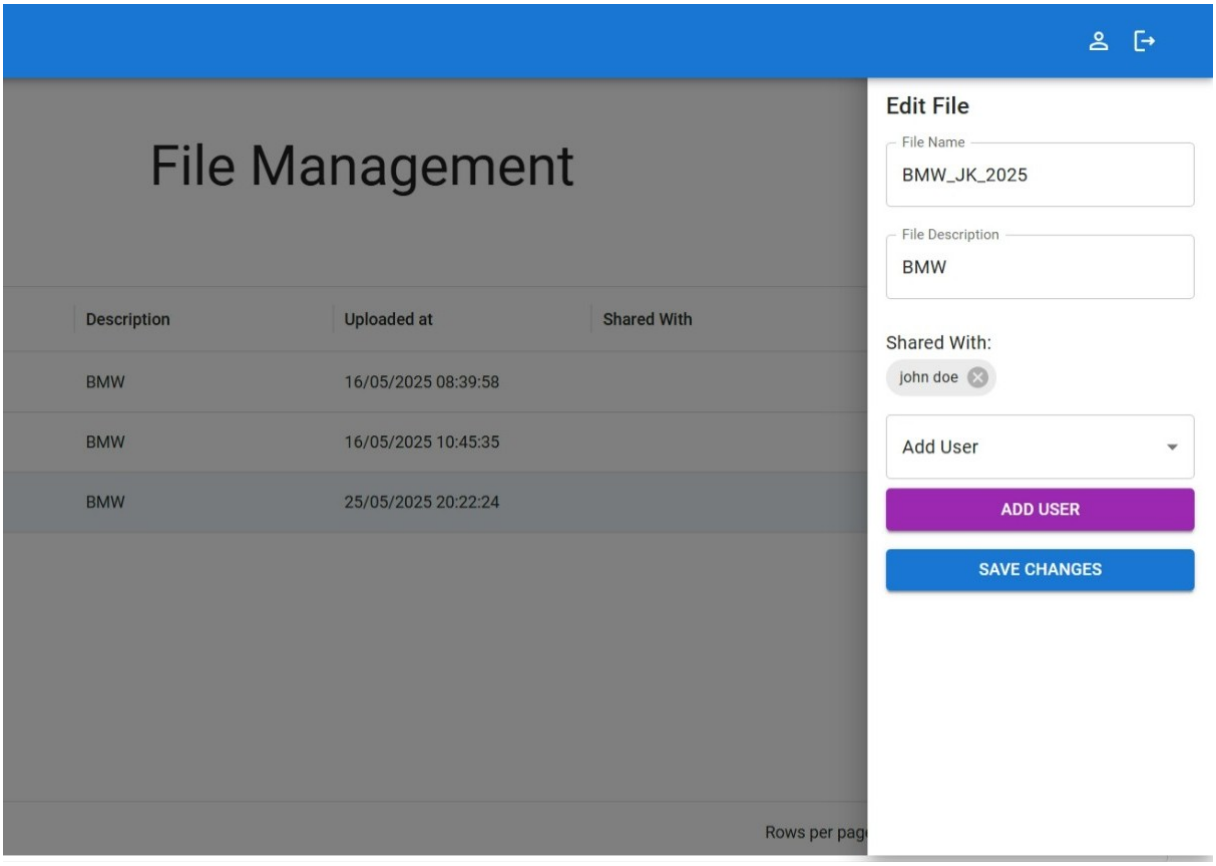

Figure 4.6: Interface for editing and sharing files.

4.4.4 Ethernet Topology Process

From this dashboard, the user can consult the diagram topology of either CAN or Ethernet, depending on the type of the file. After clicking on "View Topology," the application renders a tree list containing all the ECUs from the database, along with their consumed and provided services, as illustrated in Figure 4.7.

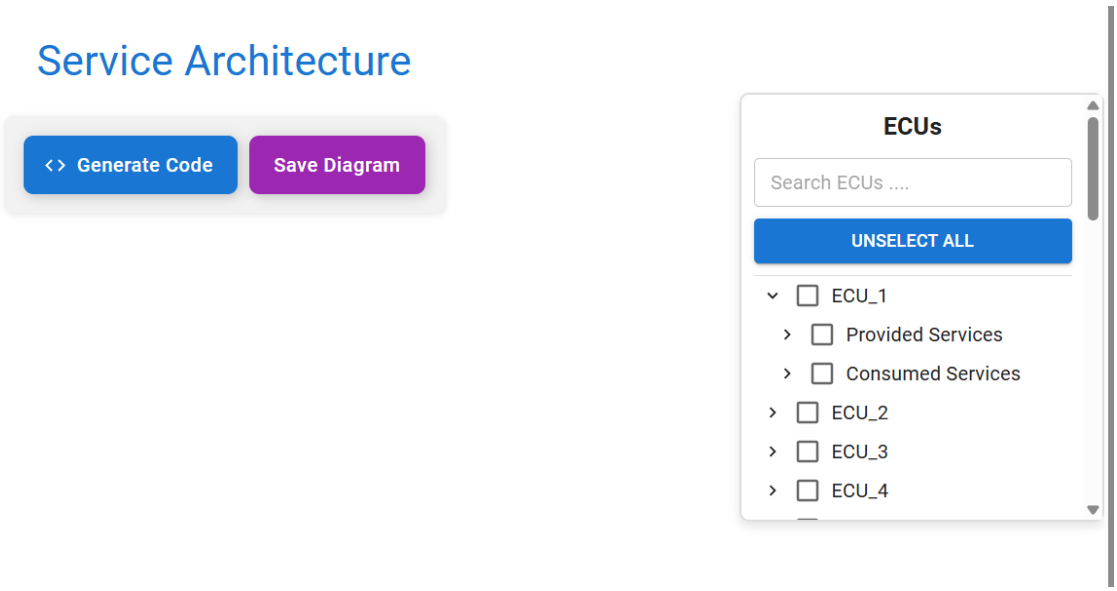

Figure 4.7: Tree list displaying all ECUs with their consumed and provided services.

The tester can select an ECU and either a provided or consumed service, prompting the page to render a graph that visually represents the selected ECU and service. The application then illustrates the relationship between these elements, as shown in Figure 4.8.

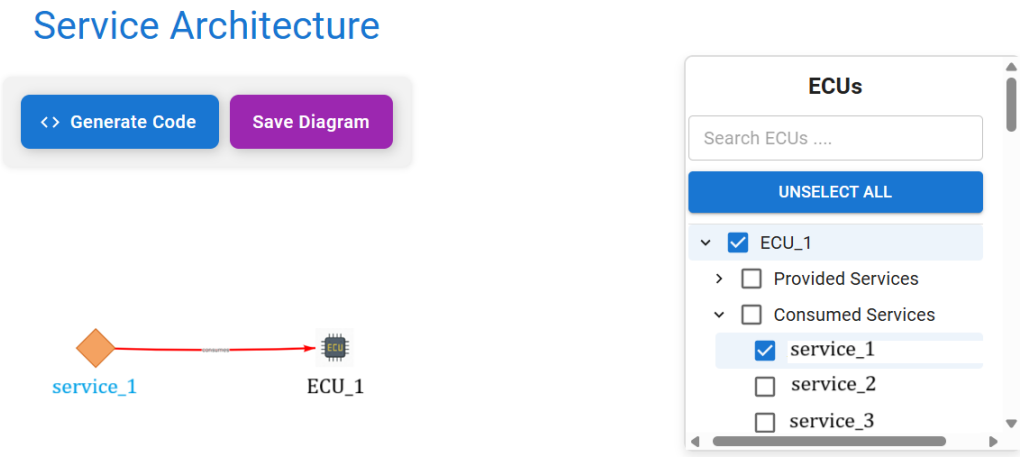

Figure 4.8: Graph illustrating the relationship between the selected ECU and service.

By clicking on the selected service within the graph, the tester can explore either the provider or the consumer of that service, as demonstrated in Figure 4.9.

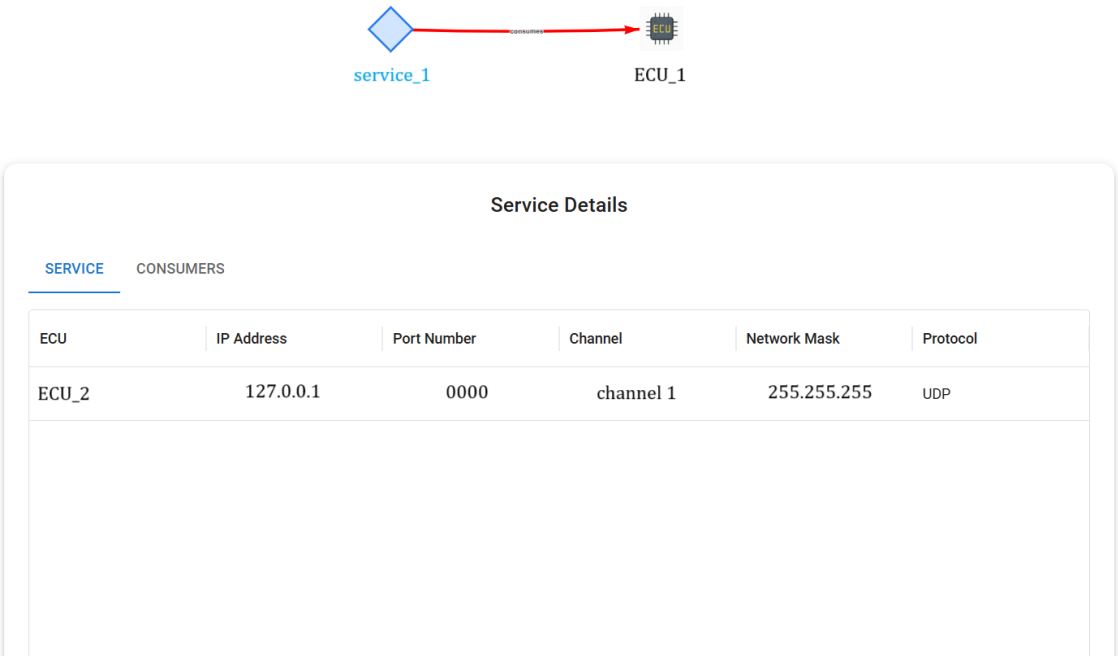

Figure 4.9: Interface showing the provider or consumer details of the selected service.

The tester can then search for and select another ECU to further visualize the consumer -provider relationships. This process can be repeated to complete the mapping of all providers and consumers of a given service. Finally, the tester can save the results or generate code based on the completed schema, as shown in Figure 4.10.

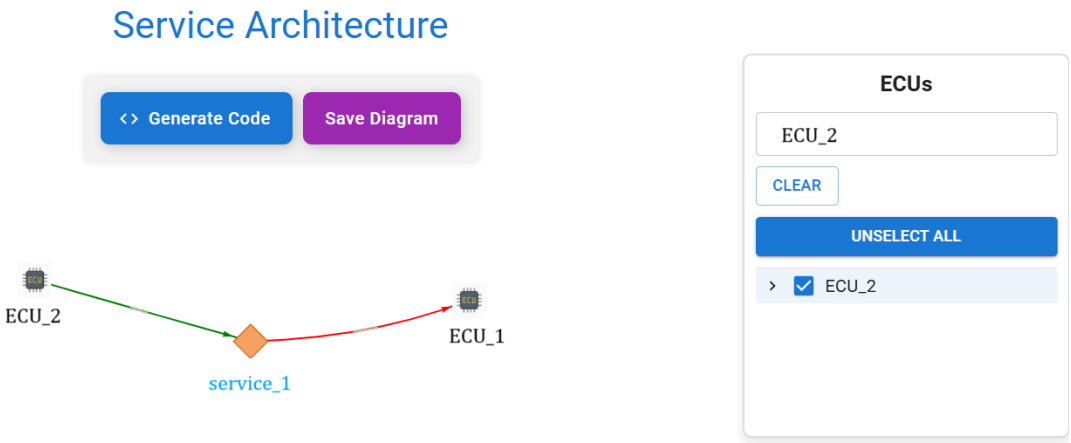

Figure 4.10: Complete topology schema with consumer-provider relationships.

4.4.5 Ethernet Code Generation Process

After the tester clicks the "Generate Code" button, they can view all the generated code and select a specific code segment, as shown in Figure 4.11.

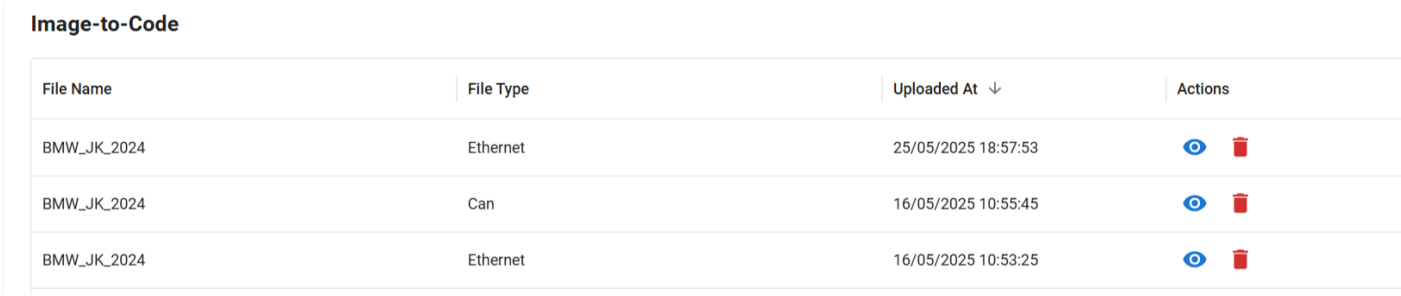

Figure 4.11: Interface showing all generated code segments.

Once a code segment is selected, the corresponding consumer-provider relationships are displayed, as illustrated in Figure 4.12.

Figure 4.12: Visualization of consumer-provider relationships with corresponding code.

The final image, shown in Figure 4.13, displays the actual code. This code contains functions for sending and receiving messages using an IP-based protocol. The tester has the flexibility to modify the payload and the MAC address as needed.

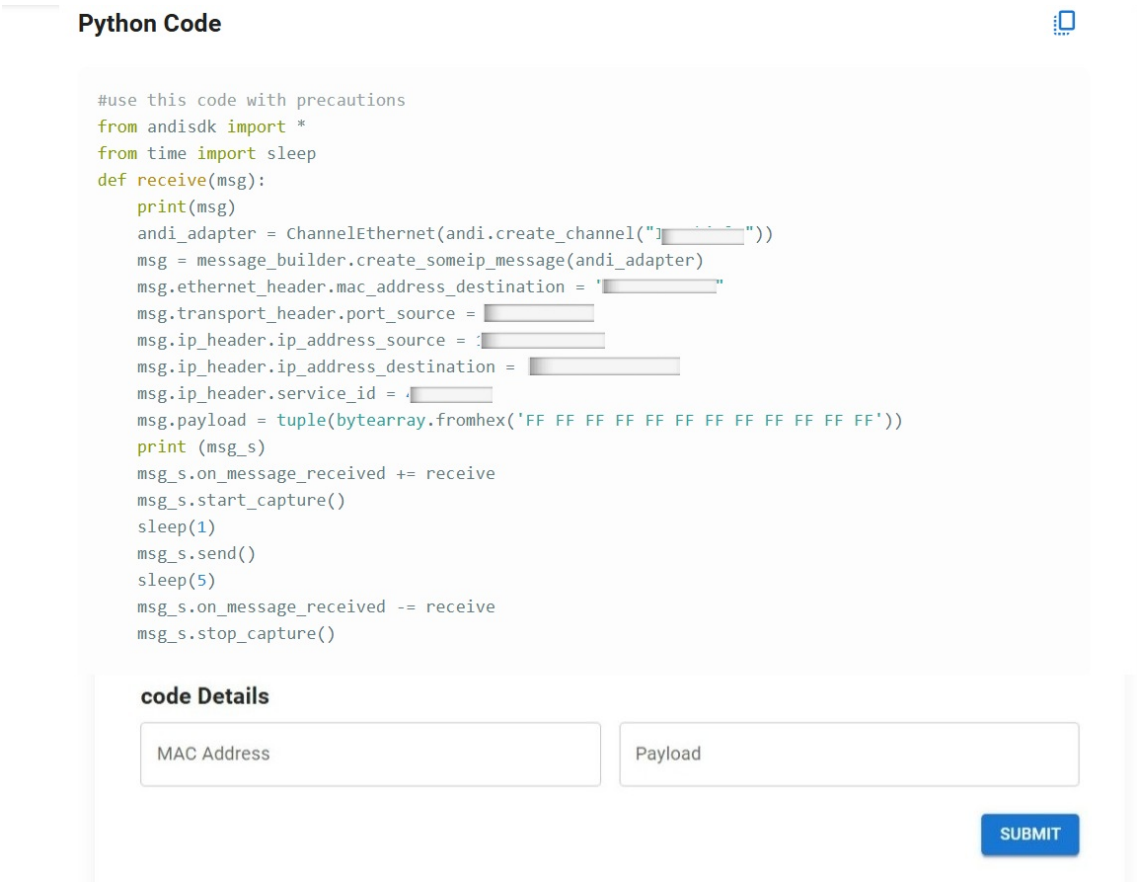

Figure 4.13: Generated code for Ethernet communication.

4.4.6 CAN Topology Process

Similar to the Ethernet process, the CAN topology process involves drawing the relationships between each ECU's input and output. The tester follows the same steps as described in the Ethernet process, as illustrated in Figure 4.14.

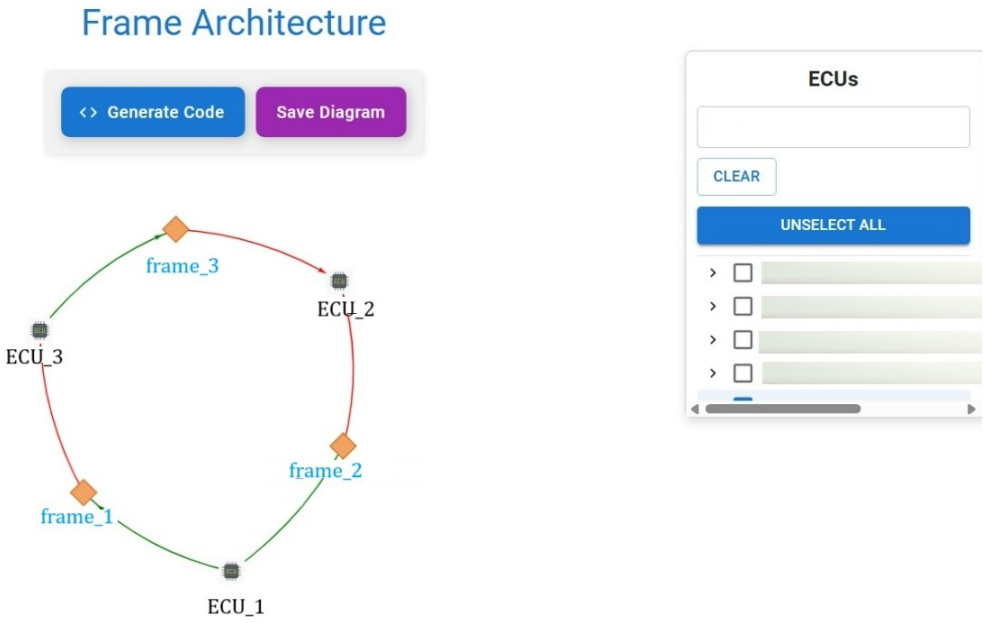

Figure 4.14: CAN topology diagram showing relationships between ECUs.

After creating the diagram, the tester can also view the properties of the CAN frame, such as the frame ID and frame length. These details are shown in Figure 4.15.

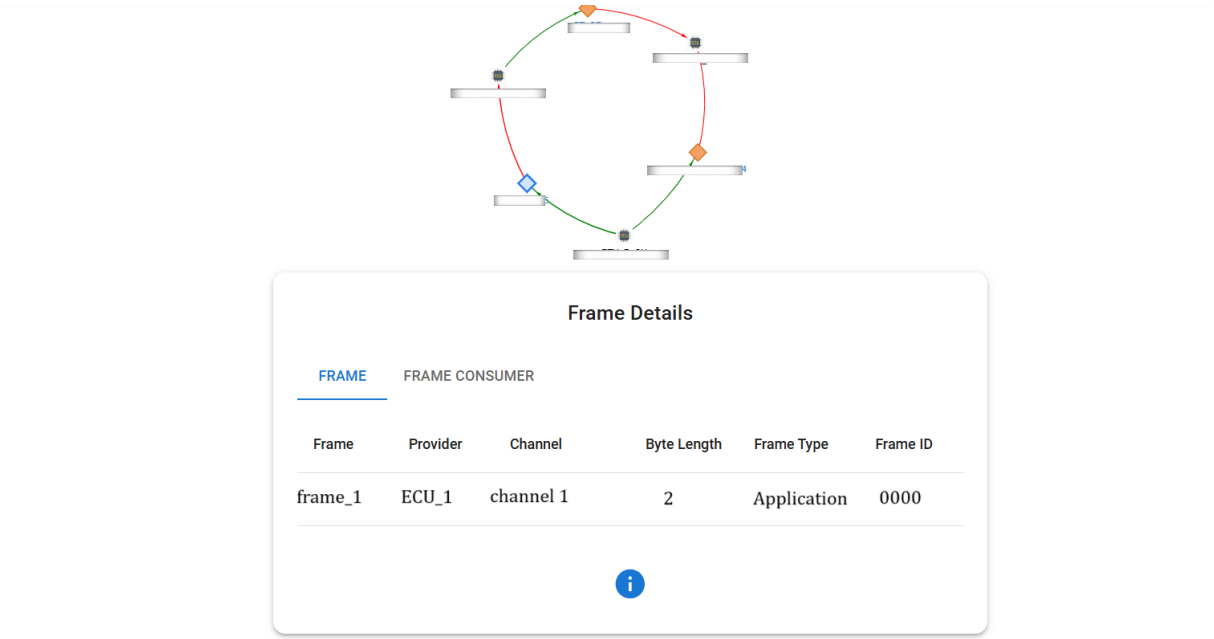

Figure 4.15: CAN frame properties including frame ID and length.

4.4.7 CAN Code Generation Process

The tester can also generate code for the CAN protocol, following a process similar to that of the Ethernet protocol. After clicking on "View Code", the tester will see an image displaying all the relationships between frames and ECUs, along with the corresponding code, as shown in Figure 4.16.

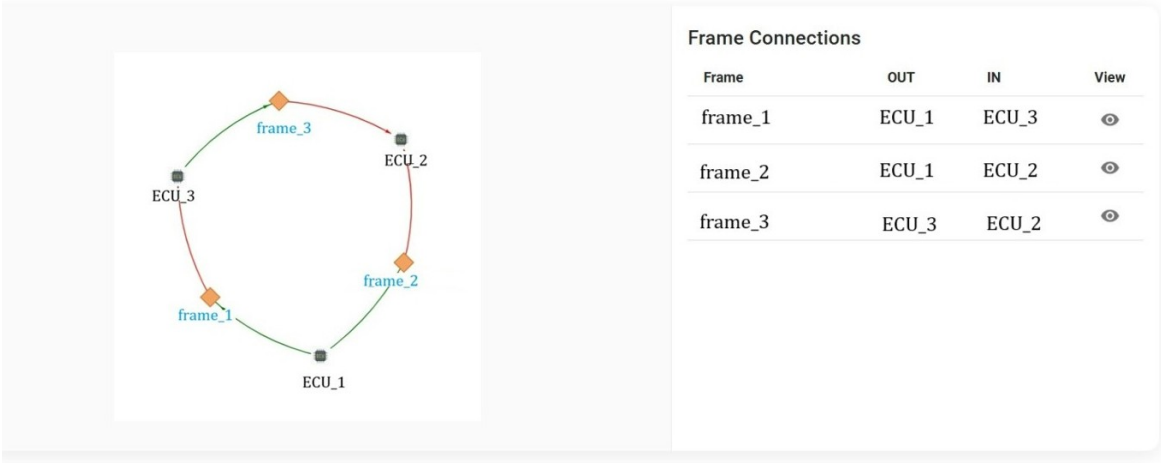

Figure 4.16: Overview of CAN frames and ECU relationships with corresponding code.

Next, the tester can view the generated code, which includes a function to handle the transmission of CAN messages. The tester has the ability to modify or select the signals associated with the chosen frame, as well as set the raw values, as shown in Figure 4.17.

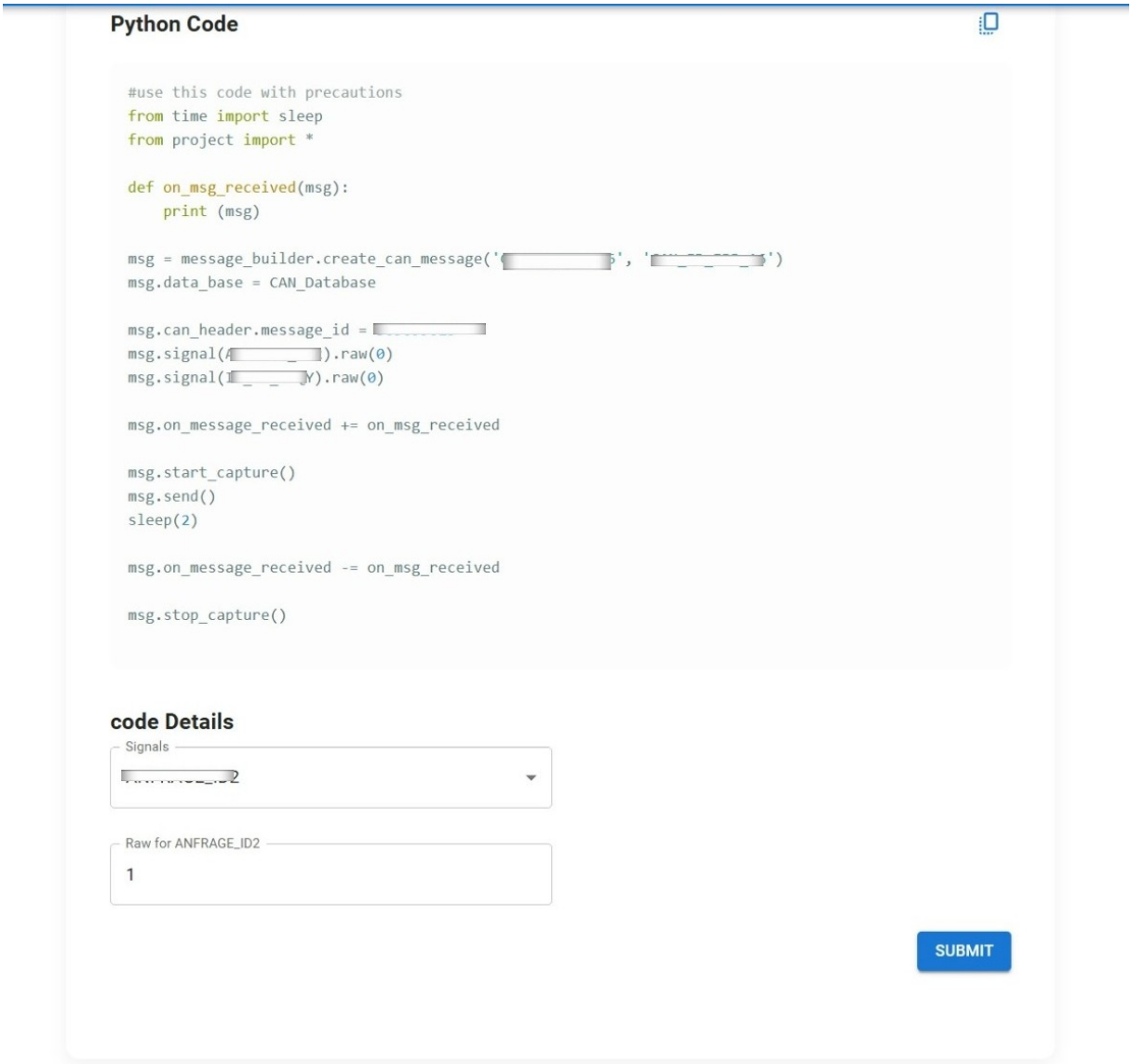

Figure 4.17: Generated CAN code with configurable signals and values.

4.4.8 Stats Page

On this page, the tester can view information about their activity within the web application, such as the number of uploaded files categorized by type, as shown in Figure 4.18.

Figure 4.18: Statistics showing the number of uploaded files by type.

By selecting a file from the list at the top of the page, the tester can also review any remaining services or frames that have not yet been tested. Additionally, the page displays graphs showing the number of uploaded code files per day, as illustrated in Figure 4.19.

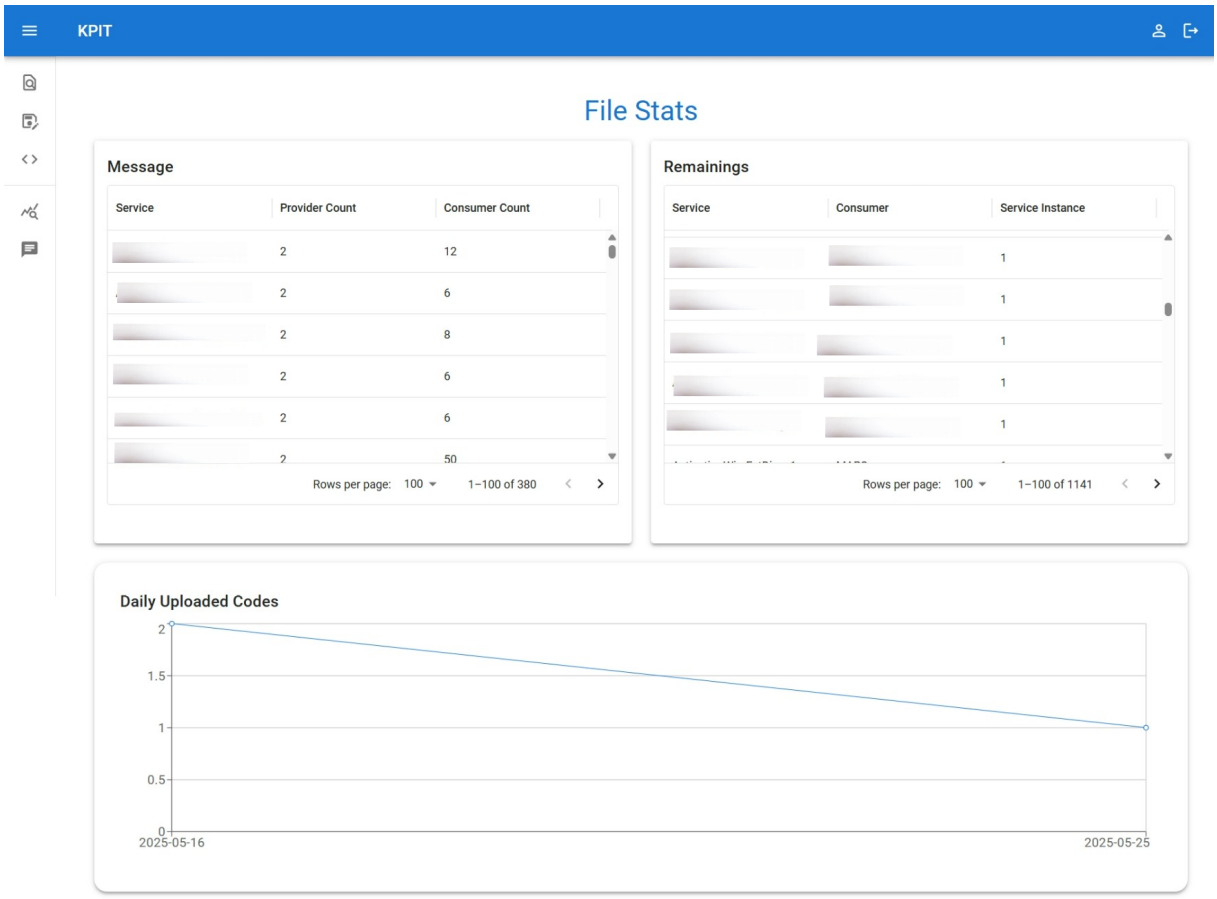

Figure 4.19: Graph showing daily uploaded code statistics.

4.4.9 Chatbot Page

After selecting the desired files, the tester can interact with the chatbot to inquire about frame or service properties, or related ECUs. In the example shown in Figure 4.21, the tester asks the chatbot to list frameID and frame type sended by the Ecu Info-1

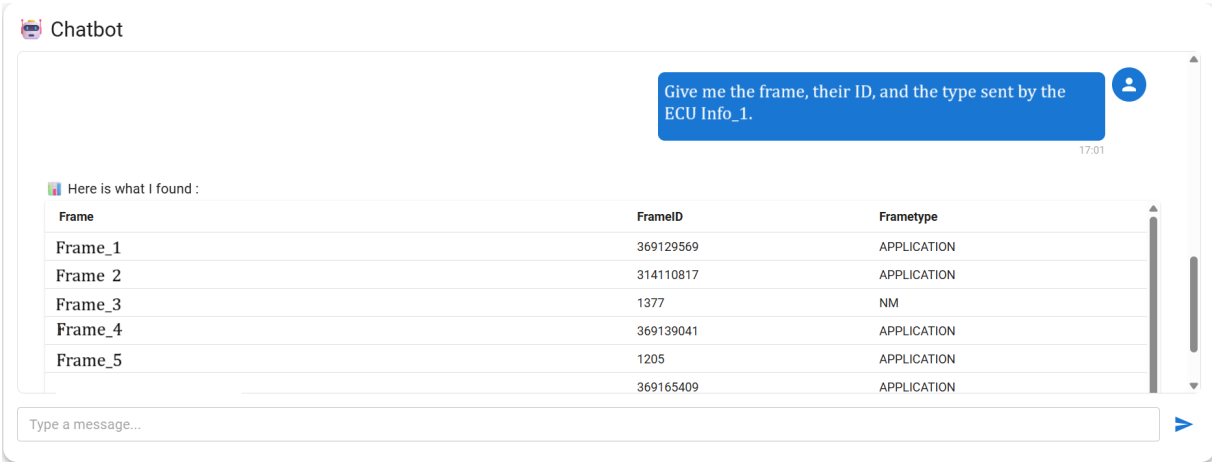

Figure 4.20: Chatbot responding to a query about frames sent by ECU Info-1.

4.4.10 Dashboard Page for Admin

The application provides a dashboard interface for the administrator, allowing them to manage tester accounts effectively, as illustrated in Figure4.21.

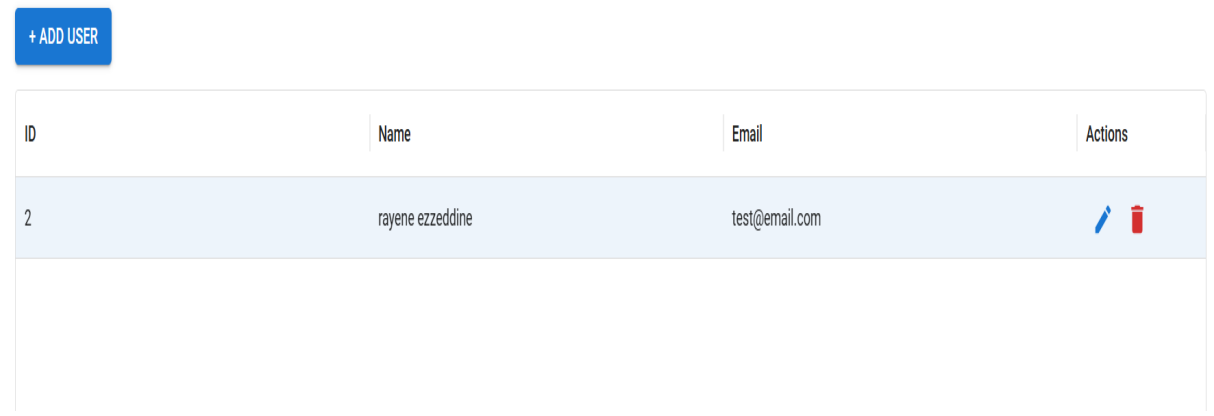

Figure 4.21: Admin dashboard interface .

4.5 Conclusion

In conclusion, this chapter has detailed the successful implementation of the system, highlighting the technologies and processes involved in realizing both the backend and frontend components. The integration of a chatbot to interpret natural language into SQL queries adds significant usability, while the intuitive interface ensures smooth navigation and operation for users. Together,

these components culminate in a fully functional and interactive platform that meets the intended system requirements outlined in earlier chapters.

GENERAL CONCLUSION

This report outlines the development of a web application designed to manage databases used in automotive communication protocols. It details our project goals, the technologies implemented, and the various phases of development.

The application features an intuitive user interface, robust data manipulation tools, and clear visualizations of Ethernet and CAN cluster configurations. It streamlines database management, enhances operational efficiency, and saves users significant time in their daily workflows.

Additionally, it includes a chat function powered by a large language model (LLM), allowing users to query database-related information conversationally.

During development, we encountered several challenges, mainly due to the complexity of the data and the requirement for a practical, high-performing solution. By following a structured approach and selecting appropriate technologies, we successfully navigated these difficulties.

This final-year project provided us with hands-on experience in web development, database management, and collaborative work using tools like GitLab. It also sharpened our skills in Python and JavaScript, and deepened our proficiency with frameworks such as Django and React.

In conclusion, our application contributes to the enhancement of data management practices within the automotive communication domain. We hope it will deliver real value to KPIT and support the daily operations of testing teams.

Looking ahead, the application lays a solid foundation for future developments. These include integration with tools like PyShark to interpret generated code outputs, and the integration of AI agents to assist with test planning and streamline the overall testing process.

Bibliography

[1] **KPIT**, *KPIT's logo*, [Online]. Available: https://www.entreprises-magazine.com/wp-content/uploads/2025/04/KPIT-Tunisie-1024x576.jpg [Accessed: May 2, 2025].

[2] **Fulcrum**, *Software Testing Life Cycle*, [Online]. Available: https://fulcrum.rocks/blog/software-testing-life-cycle [Accessed: June 1, 2025].

[3] **Flexi Project**, *Agile Methodologies A Guide to Flexible Project Management*, [Online]. Available: https://flexi-project.com/ [Accessed: May 2, 2025].

[4] **Wimi Teamwork**, *SCRUM Agile Methodology*, [Online]. Available: https://www.wimi-teamwork.com/static/medias/methode-scrum.png [Accessed: May 2, 2025].

[5] **Scrum.org** ,*Scrum Guide*, [Online]. Available: https://scrumguides.org/scrum-guide.html [Accessed: Mar. 18, 2025].

[6] **Jabil**, *Automotive Industry Trends – Prepared for the Evolution*, [Online]. Available: https://www.jabil.com/blog/automotive-technology-trends.html [Accessed: Mar. 18, 2025].

[7] **PwC**, *Automotive Industry Trends*, [Online]. Available: https://www.pwc.com/us/en/industries/industrial-products/library/automotive-industry-trends.html [Accessed: Mar. 18, 2025].

[8] **Softing**, *Automotive Protocols*, [Online]. Available: https://automotive.softing.com/ [Accessed: Mar. 18, 2025].

[9] **AUTOSAR**, *AUTOSAR (AUTomotive Open System ARchitecture)*, [Online]. Available: https://www.autosar.org/ [Accessed: Mar. 19, 2025].

[10] **Embitel**, *Embedded Blog*, [Online]. Available: https://www.embitel.com/blog/embedded-blog/ [Accessed: Mar. 20, 2025].

[11] **eInfochips**, *AUTOSAR in Automotive Industry*, [Online]. Available: https://www.einfochips.com/blog/autosar-in-automotive-industry/ [Accessed: Mar. 20, 2025].

[12] **Embitel**, *Powertrain Management, Body Electronics, and Infotainment Systems*, [Online]. Available: https://www.embitel.com/blog/embedded-blog/ [Accessed: Mar. 20, 2025].

[13] Vector Informatik GmbH ,"Ethernet SOME IP Replay",*Application Note AN.IND.1.25*, Oct 2022

[14] **COVESA**, *SomeIP Message*, [Online]. Available: https://github.com/COVESA/vsomeip/wiki/vsomeip-in-10-minutes [Accessed: Apr. 5, 2025].

[15] **Supermicro**, *CAN Bus*, [Online]. Available: https://www.supermicro.com/en/glossary/can-bus [Accessed: Apr. 5, 2025].

[16] Steve Corrigan ,"Controller Area Network (CAN) Implementation Guide", *Application Report-SLOA101B*, May 2016

[17] Vector Informatik GmbH, "CAN FD Introduction.", *Application Note AN.IND.1.25*, Oct.2022.

[18] **MDPI**, *CAN and CAN FD Data Frame Formats*, [Online]. Available: https://www.mdpi.com/IoT/IoT-05-00015/ [Accessed: May 5, 2025].

[19] **Dataloop**, *Defog SQLCoder*, [Online]. Available: https://dataloop.ai/library/model/defog_sqlcoder [Accessed: Jun. 6, 2025].

[20] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and Efficient Foundation Language Models", *arXiv preprint arXiv:2302.13971*, Feb. 2023.

[21] **Hugging Face**, *Defog SQLCoder-7B-2*, [Online]. Available: https://huggingface.co/defog/sqlcoder-7b-2 [Accessed: Jun. 6, 2025].

[22] **Hugging Face**, *Performance Comparison Between SQLCoder and Other LLMs*, [Online]. Available: https://cdn-uploads.huggingface.co/production/uploads [Accessed: Jun. 6, 2025].

[23] **Django Software Foundation**, *The Django Project*, [Online]. Available: https://www.djangoproject.com/ [Accessed: Jun. 6, 2025].

[24] **GeeksforGeeks**, *What is React?*, [Online]. Available: https://www.geeksforgeeks.org/what-is-react/ [Accessed: Jun. 6, 2025].

[25] **GeeksforGeeks**, *How Tokenizing Text, Sentence, and Words Works in NLP*, [Online]. Available: https://www.geeksforgeeks.org/ [Accessed: Jun. 6, 2025].

[26] **Omkar**, *Introduction to GGML*, [Online]. Available: https://omkar.xyz/intro-ggml/ [Accessed: Jun. 6, 2025].

Web Application to Visualize Network toplogy

Ezzedddine Rayene

Abstract :

As part of my final-year project for the National Engineering Degree in Electronics and Communication, wedeveloped a software application for KPIT to optimize the management and analysis of large automotive communication protocol databases. To assist testers, we implemented a user-friendly interface with diagrams and a chatbot, enhancing data understanding and efficiency. The project was built using Visual Studio with a Django back-end, React front-end, and languages like Python and JavaScript.

Key Words : Django, React, Communication Protocols, Software Development, Software Testing, CAN, NLP, SQL Coder.

Résumé :

Dans le cadre de mon projet de fin d'études pour l'obtention du Diplôme National d'Ingénieur en Génie Électronique et Communication, nous avons développé une application logicielle pour l'entreprise KPIT. Ce projet vise à optimiser la gestion et l'analyse de grandes bases de données regroupant des protocoles de communication automobile. Pour aider les testeurs, une interface graphique avec diagrammes et chatbot a été créée, améliorant la compréhension des données et l'efficacité du travail. Le développement s'est appuyé sur Visual Studio, avec une architecture Django (back-end), React (front-end), et les langages Python et JavaScript.

Mots Clés : Django, React, Protocoles de communication, Développement logiciel, Tests logiciels, CAN, NLP, SQL Coder.

تلخيص:

في إطار مشروع تخرّج لنيل شهادة مهندس في هندسة الإلكترونيات والاتصالات، قمنا بتصميم وتطوير تطبيق برمجي لفائدة شركة KPIT.يهدف هذا المشروع الى تحسين إدارة وتحليل قواعد البيانات الضخمة التي تحتوي على بروتوكولات الاتصال في السيارات. ولتسهيل عمل المختبرين تم تنفيذ واجهة رسومية تفاعلية تتضمن مخططات وربوت محادثة، مما ساهم في تعزيز فهم البيانات ورفع كفاءة العمل. وقد تم تطوير المشروع باستخدام بيئة VS، مع اعتماد بنية تجمع بين إطار Django للواجهة الخلفية وReact للواجهة الامامية، وذلك باستخدام لغتي البرمجة Python وJS.

الكلمات المفتاحية: Django ، بروتوكولات الاتصال ، تطوير البرمجيات ، اختبار البرمجيات، معالجة اللغة الطبيعية NLP, برمجة SQL .